# AI Applications Lecture 13

## Iterative Latent Generation: Stable Diffusion 1.5 U-Net and Qwen-Image-2.1 DiT

SUZUKI, Atsushi[a]

[a] I thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

# Outline

1. Introduction

2. Mathematical notation

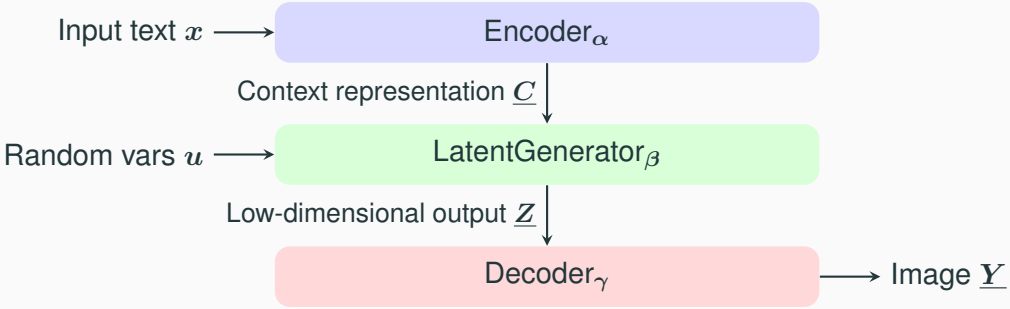
3. What are the necessary conditions for NNs in iterative generation?

4. How does Stable Diffusion 1.5's U-Net work?

5. Diffusion Transformer of Qwen-Image-2.1

6. Summary

## Review the latent generation framework




Example iterative generation: Let $w_0 = \epsilon$. For $k = 0, 1, ..., K-1$:

- **Reverse diffusion (DDIM)**:
  $w_{k+1} \leftarrow \frac{b(t_{k+1})}{b(t_k)} w_k + \left[ a(t_{k+1}) - \frac{a(t_k) b(t_{k+1})}{b(t_k)} \right] \widehat{\epsilon}_\theta(w_k, c, t_k)$.
- **Flow matching (linear path + Euler)**: $w_{k+1} \leftarrow w_k + (t_{k+1} - t_k) \widehat{v}_\theta(w_k, c, t_k)$.

# Learning outcomes

At the end of this lecture, you will be able to:

## Learning outcomes

At the end of this lecture, you will be able to:

1. **Judge** what types of neural networks can apply to iterative generation with a user input.

## Learning outcomes

At the end of this lecture, you will be able to:

1. **Judge** what types of neural networks can apply to iterative generation with a user input.
2. **Explain** pros and cons of U-net and diffusion transformers.

# Outline


**1. Introduction**

2. Mathematical notation

3. What are the necessary conditions for NNs in iterative generation?

4. How does Stable Diffusion 1.5's U-Net work?

5. Diffusion Transformer of Qwen-Image-2.1

6. Summary

# Outline

1. Introduction

**2. Mathematical notation**

3. What are the necessary conditions for NNs in iterative generation?

4. How does Stable Diffusion 1.5's U-Net work?

5. Diffusion Transformer of Qwen-Image-2.1

6. Summary

# Preliminaries: mathematical notations

**Set:**

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} \mid P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $[1, k]_{\mathbb{Z}} \coloneqq \{1, \ldots, k\}$

**Function:**

- $f : \mathcal{X} \to \mathcal{Y}$: $f$ maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of $f$ for input $x$.

**Definition:**

- $(\mathrm{LHS}) \coloneqq (\mathrm{RHS})$: Left side is defined by the right side.

# Preliminaries: mathematical notations

**Sequence:** Denoted by $\boldsymbol{a} = (a_1, a_2, \dots)$.

- A function $\boldsymbol{a} : [1, n]_{\mathbb{Z}} \to \mathcal{A}$.
- Length is denoted by $|\boldsymbol{a}|$.

**Vector:** Denoted by $\boldsymbol{v}$.

- A column of numbers, $\boldsymbol{v} \in \mathbb{R}^n$.
- $i$-th element is $v_i$.

**Matrix:** Denoted by $\boldsymbol{A}$.

- $m \times n$ matrix: $\boldsymbol{A} \in \mathbb{R}^{m,n}$.
- $(i, j)$-th element is $a_{i,j}$.
- Transpose: $\boldsymbol{A}^\top$.

**Tensor:** Denoted by $\underline{\boldsymbol{A}}$.

- Simply a multi-dimensional array.
- Vector $\to$ 1st-order, Matrix $\to$ 2nd-order.

# Outline

1. Introduction

**2. Mathematical notation**

3. What are the necessary conditions for NNs in iterative generation?

4. How does Stable Diffusion 1.5's U-Net work?

5. Diffusion Transformer of Qwen-Image-2.1

6. Summary

# Outline

1. Introduction

2. Mathematical notation

**3. What are the necessary conditions for NNs in iterative generation?**

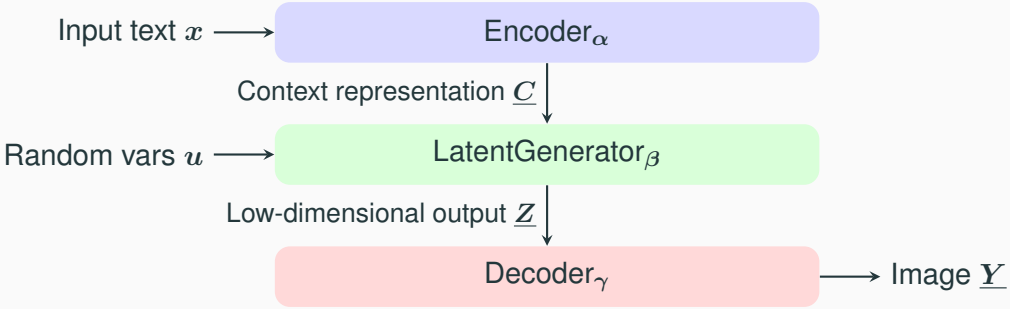
4. How does Stable Diffusion 1.5's U-Net work?

5. Diffusion Transformer of Qwen-Image-2.1

6. Summary

**Question**: A NN used in iterative generation is the center part. It must accept the user input information, must work in the iterative generation framework, and must output latent data to the VAE.

## Review the latent generation framework




Example iterative generation: Let $w_0 = \epsilon$. For $k = 0, 1, ..., K-1$:

- **Reverse diffusion (DDIM)**:
  $w_{k+1} \leftarrow \frac{b(t_{k+1})}{b(t_k)} w_k + \left[ a(t_{k+1}) - \frac{a(t_k) b(t_{k+1})}{b(t_k)} \right] \widehat{\epsilon}_\theta(w_k, c, t_k)$.
- **Flow matching (linear path + Euler)**: $w_{k+1} \leftarrow w_k + (t_{k+1} - t_k) \widehat{v}_\theta(w_k, c, t_k)$.

## Necessary conditions for a NN to be used in the iterative generation framework

From the input and output aspects, a NN needs to satisfy some necessary conditions.

- The NN ($\widehat{\epsilon}_\theta$ or $\widehat{v}_\theta$) must accept the representation of the user input $\underline{\boldsymbol{C}}$ as well as the input $\underline{\boldsymbol{Z}}_k$ and return the output $\underline{\boldsymbol{Z}}_{k+1}$.

**Exercise**

*What are other necessary conditions for a NN to be used in the iterative (visual data) generation framework, especially concerning the input-output shape?*

## Necessary conditions for a NN to be used in the iterative generation framework

**Answer**:

## Necessary conditions for a NN to be used in the iterative generation framework

**Answer**:

- **The input shape and output shape must be always identical**.

## Necessary conditions for a NN to be used in the iterative generation framework

**Answer**:

- **The input shape and output shape must be always identical**. When we input $\underline{\boldsymbol{W}}_k \in \mathbb{R}^{64,64,4}$ to $\underline{\widehat{\boldsymbol{V}}}_\theta$, then the output size $\underline{\boldsymbol{W}}_{k+1} = \underline{\widehat{\boldsymbol{V}}}_\theta(\underline{\boldsymbol{W}}_k, \underline{\boldsymbol{C}}, t_k)$ should satisfy

## Necessary conditions for a NN to be used in the iterative generation framework

**Answer**:

- **The input shape and output shape must be always identical**. When we input $\underline{\boldsymbol{W}}_k \in \mathbb{R}^{64,64,4}$ to $\underline{\widehat{\boldsymbol{V}}}_\theta$, then the output size $\underline{\boldsymbol{W}}_{k+1} = \underline{\widehat{\boldsymbol{V}}}_\theta(\underline{\boldsymbol{W}}_k, \underline{\boldsymbol{C}}, t_k)$ should satisfy $\underline{\boldsymbol{W}}_{k+1} \in \mathbb{R}^{64,64,4}$.

## Necessary conditions for a NN to be used in the iterative generation framework

**Answer**:

- **The input shape and output shape must be always identical**. When we input $\underline{\boldsymbol{W}}_k \in \mathbb{R}^{64,64,4}$ to $\underline{\widehat{\boldsymbol{V}}}_\theta$, then the output size $\underline{\boldsymbol{W}}_{k+1} = \underline{\widehat{\boldsymbol{V}}}_\theta(\underline{\boldsymbol{W}}_k, \underline{\boldsymbol{C}}, t_k)$ should satisfy $\underline{\boldsymbol{W}}_{k+1} \in \mathbb{R}^{64,64,4}$.
  If the input and output shapes are not identical, the iterative update does not work.

## Necessary conditions for a NN to be used in the iterative generation framework

**Answer**:

- **The input shape and output shape must be always identical**. When we input $\underline{\boldsymbol{W}}_k \in \mathbb{R}^{64,64,4}$ to $\underline{\widehat{\boldsymbol{V}}}_\theta$, then the output size $\underline{\boldsymbol{W}}_{k+1} = \underline{\widehat{\boldsymbol{V}}}_\theta(\underline{\boldsymbol{W}}_k, \underline{\boldsymbol{C}}, t_k)$ should satisfy $\underline{\boldsymbol{W}}_{k+1} \in \mathbb{R}^{64,64,4}$.
  If the input and output shapes are not identical, the iterative update does not work. For example, the addition at the RHS of the flow matching update $w_{k+1} \leftarrow w_k + (t_{k+1} - t_k)\widehat{v}_\theta(w_k, c, t_k)$ does not work.

## Necessary conditions for a NN to be used in the iterative generation framework

**Answer**:

- **The input shape and output shape must be always identical**. When we input $\underline{\boldsymbol{W}}_k \in \mathbb{R}^{64,64,4}$ to $\underline{\widehat{\boldsymbol{V}}}_\theta$, then the output size $\underline{\boldsymbol{W}}_{k+1} = \underline{\widehat{\boldsymbol{V}}}_\theta(\underline{\boldsymbol{W}}_k, \underline{\boldsymbol{C}}, t_k)$ should satisfy $\underline{\boldsymbol{W}}_{k+1} \in \mathbb{R}^{64,64,4}$.
  If the input and output shapes are not identical, the iterative update does not work. For example, the addition at the RHS of the flow matching update $w_{k+1} \leftarrow w_k + (t_{k+1} - t_k)\widehat{v}_\theta(w_k, c, t_k)$ does not work.
- **It must accept variable shape inputs** with a fixed parameter vector.

## Necessary conditions for a NN to be used in the iterative generation framework

**Answer**:

- **The input shape and output shape must be always identical**. When we input $\underline{\boldsymbol{W}}_k \in \mathbb{R}^{64,64,4}$ to $\underline{\widehat{\boldsymbol{V}}}_\theta$, then the output size $\underline{\boldsymbol{W}}_{k+1} = \underline{\widehat{\boldsymbol{V}}}_\theta(\underline{\boldsymbol{W}}_k, \underline{\boldsymbol{C}}, t_k)$ should satisfy $\underline{\boldsymbol{W}}_{k+1} \in \mathbb{R}^{64,64,4}$.
  If the input and output shapes are not identical, the iterative update does not work. For example, the addition at the RHS of the flow matching update $w_{k+1} \leftarrow w_k + (t_{k+1} - t_k)\widehat{v}_\theta(w_k, c, t_k)$ does not work.
- **It must accept variable shape inputs** with a fixed parameter vector. This is to control the output shape. By controling the tensor shape of the input of the (VAE) Decoder, we can select the output image size.

## Necessary conditions for a NN to be used in the iterative generation framework

**Answer**:

- **The input shape and output shape must be always identical**. When we input $\underline{\boldsymbol{W}}_k \in \mathbb{R}^{64,64,4}$ to $\underline{\widehat{\boldsymbol{V}}}_\theta$, then the output size $\underline{\boldsymbol{W}}_{k+1} = \underline{\widehat{\boldsymbol{V}}}_\theta(\underline{\boldsymbol{W}}_k, \underline{\boldsymbol{C}}, t_k)$ should satisfy $\underline{\boldsymbol{W}}_{k+1} \in \mathbb{R}^{64,64,4}$.
  If the input and output shapes are not identical, the iterative update does not work. For example, the addition at the RHS of the flow matching update $w_{k+1} \leftarrow w_k + (t_{k+1} - t_k)\widehat{v}_\theta(w_k, c, t_k)$ does not work.
- **It must accept variable shape inputs** with a fixed parameter vector. This is to control the output shape. By controling the tensor shape of the input of the (VAE) Decoder, we can select the output image size.

The first requirement is different from the VAE decoder situation, while the second is the same.

# Outline

1. Introduction

2. Mathematical notation

**3. What are the necessary conditions for NNs in iterative generation?**

4. How does Stable Diffusion 1.5's U-Net work?

5. Diffusion Transformer of Qwen-Image-2.1

6. Summary

# Outline

1. Introduction

2. Mathematical notation

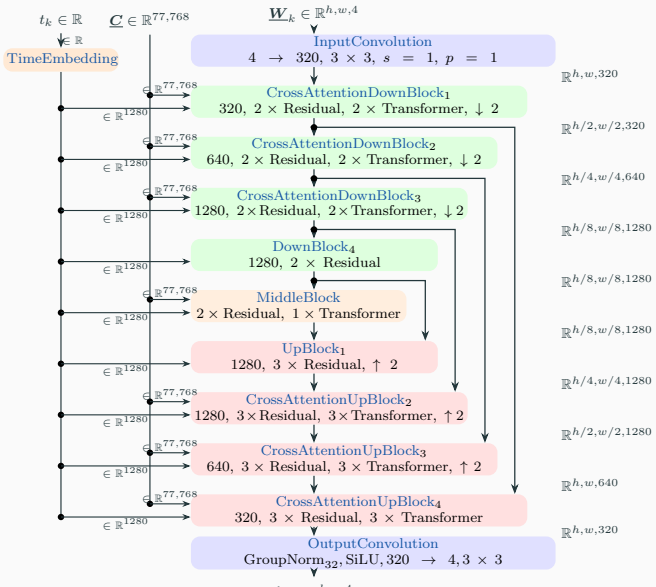
3. What are the necessary conditions for NNs in iterative generation?

**4. How does Stable Diffusion 1.5's U-Net work?**

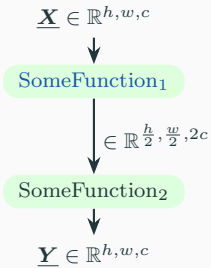
5. Diffusion Transformer of Qwen-Image-2.1

6. Summary

# The whole structure




## Legend




Rounded boxes: functions.
Arrows: tensors; the adjacent superscripts give their shapes.
Function in ThisColor indicates it has learnable parameters.

$h = H/8, w = W/8$; image dimensions are divisible by 64.

U-Net outputs predicted noise.

Source: SD 1.5 U-Net configuration; Diffusers UNet2DConditionModel.

## Which parts should we focus on?

Focus on where we integrate the current latent image's information $\underline{\boldsymbol{W}}_k$ and the user input information $\underline{\boldsymbol{C}}$.

## Which parts should we focus on?

Focus on where we integrate the current latent image's information $\underline{\boldsymbol{W}}_k$ and the user input information $\underline{\boldsymbol{C}}$.

**Exercise**

*Where?*

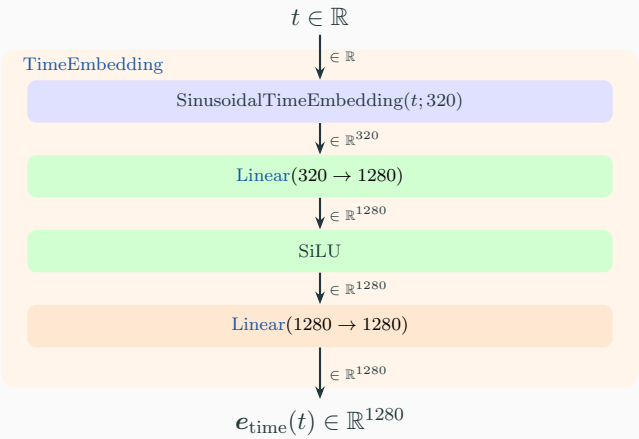
## Which parts should we focus on?

**Answer**:

## Which parts should we focus on?

**Answer**: In Stable Diffusion 1.5, **cross attention** blocks integrate the current latent image information $\underline{\boldsymbol{W}_k}$ and the user data information $\underline{\boldsymbol{C}}$.

# Time embedding and input projection




Source: SD U-Net config; Timesteps and TimestepEmbedding

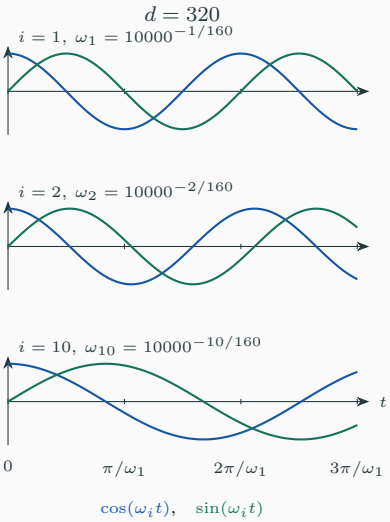
# Sinusoidal timestep embedding

$\mathrm{SinusoidalTimeEmbedding}(t; d) = \boldsymbol{e}(t) \in \mathbb{R}^d$ is defined as follows.

For even dimension $d$ and $0 \le i < d/2$, $\boldsymbol{e}(t) \in \mathbb{R}^d$ is given by

$$e_i(t) = \cos\left(t 10000^{-i/(d/2)}\right),$$
$$e_{d/2+i}(t) = \sin\left(t 10000^{-i/(d/2)}\right).$$

SD U-Net uses $d = 320$; learned time projection then maps $320 \to 1280 \to 1280$ with SiLU between the linear layers.




Source: diffusers Timesteps; TimestepEmbedding

# Sinusoidal timestep embedding — exercise

**Exercise**

*For $d = 4$ and $t = 0.01\pi$, calculate $\boldsymbol{e}(t) \in \mathbb{R}^4$ using the definition of the sinusoidal timestep embedding. Give an exact expression and approximate each component to four decimal places.*

**Hint**: Use the Maclaurin expansions

$$\cos x = 1 - \frac{x^2}{2!} + O(x^4), \qquad \sin x = x + O(x^3).$$

Source: diffusers Timesteps; TimestepEmbedding

# Sinusoidal timestep embedding — answer

**Answer**:

# Sinusoidal timestep embedding — answer

**Answer**: For $d = 4$, we have $d/2 = 2$ and $i \in \{0, 1\}$. The frequencies are $10000^0 = 1$ and $10000^{-1/2} = 1/100$. Since $t = 0.01\pi = \pi/100$,

$$\cos\left(\frac{\pi}{100}\right) = 1 - \frac{\pi^2}{2 \cdot 100^2} + O\left(\frac{\pi^4}{100^4}\right) \approx 0.9995,$$

$$\cos\left(\frac{\pi}{10000}\right) = 1 - \frac{\pi^2}{2 \cdot 10000^2} + O\left(\frac{\pi^4}{10000^4}\right) \approx 1.0000,$$

$$\sin\left(\frac{\pi}{100}\right) = \frac{\pi}{100} + O\left(\frac{\pi^3}{100^3}\right) \approx 0.0314,$$

$$\sin\left(\frac{\pi}{10000}\right) = \frac{\pi}{10000} + O\left(\frac{\pi^3}{10000^3}\right) \approx 0.0003.$$

$$\boldsymbol{e}(0.01\pi) = \begin{pmatrix} \cos(\pi/100) \\ \cos(\pi/10000) \\ \sin(\pi/100) \\ \sin(\pi/10000) \end{pmatrix} \approx \begin{pmatrix} 0.9995 \\ 1.0000 \\ 0.0314 \\ 0.0003 \end{pmatrix}.$$

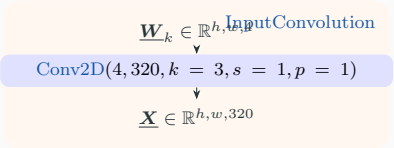


Source: diffusers Timesteps; TimestepEmbedding

# InputConvolution

The first U-Net operation is one learned 2D convolution. Let $h = H/8$, $w = W/8$.

$$\underline{\boldsymbol{W}}_k \in \mathbb{R}^{h,w,4} \xrightarrow{\text{Conv2D}} \underline{\boldsymbol{X}} \in \mathbb{R}^{h,w,320}.$$



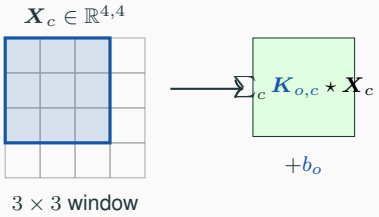

- Kernel $3 \times 3$, stride $1$, padding $1$, dilation $1$.
- Bias: learned, $320$ scalars.
- Weight: $\underline{\boldsymbol{K}} \in \mathbb{R}^{3,3,4,320}$.

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Two-dimensional convolution

Let $\underline{\boldsymbol{X}} = (\boldsymbol{X}_1, \ldots, \boldsymbol{X}_{C_\text{in}}) \in \mathbb{R}^{h,w,C_\text{in}}$ with $\boldsymbol{X}_c \in \mathbb{R}^{h,w}$. For every output channel $o$,

$$\boldsymbol{Y}_o = \sum_{c=1}^{C_\text{in}} \boldsymbol{K}_{o,c} \star \boldsymbol{X}_c + b_o \mathbf{1}.$$




Here $\boldsymbol{K}_{o,c} \in \mathbb{R}^{k,k}$; $\star$ denotes 2D cross-correlation with stride $s$ and padding $p$ (the operation called convolution in PyTorch). For zero-based indices,

$$Y_{u,v,o} = \sum_{c=1}^{C_\text{in}} \sum_{a,b=0}^{k-1} K_{a,b,c,o} \widetilde{X}_{su+a-p,sv+b-p,c} + b_o.$$

$\underline{\boldsymbol{Y}} \in \mathbb{R}^{h',w',C_\text{out}}$; $h' = \lfloor (h + 2p - k)/s \rfloor + 1$, similarly $w'$.

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Two-dimensional convolution — exercise

## Exercise

*Let* $\boldsymbol{X} \in \mathbb{R}^{3,3}$ *and* $\boldsymbol{K} \in \mathbb{R}^{2,2}$ *be*

$$\boldsymbol{X} = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{pmatrix}, \qquad \boldsymbol{K} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

*For* $s = 1$, $p = 0$, $C_{\text{in}} = C_{\text{out}} = 1$ *and* $b = 0$, *compute every element of* $\boldsymbol{Y}$.

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Two-dimensional convolution — answer

**Answer**:

# Two-dimensional convolution — answer

**Answer**: The output shape is $(3-2+1)\times(3-2+1)=2\times 2$. The four window products are

$$
\begin{aligned}
Y_{1,1} &= 1\cdot 1+2\cdot 0+0\cdot 0+1\cdot 1=2,\\
Y_{1,2} &= 2\cdot 1+0\cdot 0+1\cdot 0+2\cdot 1=4,\\
Y_{2,1} &= 0\cdot 1+1\cdot 0+1\cdot 0+0\cdot 1=0,\\
Y_{2,2} &= 1\cdot 1+2\cdot 0+0\cdot 0+1\cdot 1=2.
\end{aligned}
$$

Hence $\boldsymbol{Y}=\begin{pmatrix}2 & 4\\ 0 & 2\end{pmatrix}\in\mathbb{R}^{2,2}$.

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

## Pros and cons of convolutional layer

The convolutional layer can only gather a local information at once.

## Pros and cons of convolutional layer

The convolutional layer can only gather a local information at once.

To compensate for this locality, U-net apply the convolutional operation at an extremely low dimensional situation.

## Pros and cons of convolutional layer

The convolutional layer can only gather a local information at once.

To compensate for this locality, U-net apply the convolutional operation at an extremely low dimensional situation.

For example, if the original image size is $512 \times 512$, then the latent image shape handled by $\mathrm{MiddleBlock}$ is $8x8$. Each pixel they covers $64 \times 64$ pixel area. Hence, a convolution effect is not localized.

## Pros and cons of convolutional layer

The convolutional layer can only gather a local information at once.

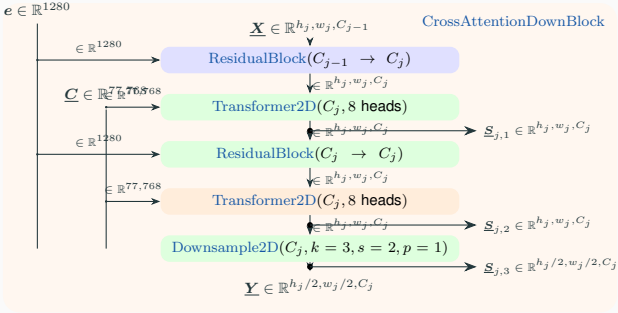
To compensate for this locality, U-net apply the convolutional operation at an extremely low dimensional situation.

For example, if the original image size is $512 \times 512$, then the latent image shape handled by $\mathrm{MiddleBlock}$ is $8x8$. Each pixel they covers $64 \times 64$ pixel area. Hence, a convolution effect is not localized.

One advantage is the computational complexity. It is $O(hwkk)$, which is acceptable.

# CrossAttentionDownBlock

For $j = 1, 2, 3$, set $C_j = (320, 640, 1280)_j$, $C_0 = 320$, $h_j = h/2^{j-1}$, $w_j = w/2^{j-1}$. Input $\underline{\boldsymbol{X}} \in \mathbb{R}^{h_j, w_j, C_{j-1}}$, time $\boldsymbol{e} \in \mathbb{R}^{1280}$, text $\underline{\boldsymbol{C}} \in \mathbb{R}^{77,768}$.
Later, $\underline{\boldsymbol{S}}_{j,1}, \underline{\boldsymbol{S}}_{j,2} \in \mathbb{R}^{h_j, w_j, C_j}$ and $\underline{\boldsymbol{S}}_{j,3} \in \mathbb{R}^{h_j/2, w_j/2, C_j}$ will be concatinated in an Up blocks.



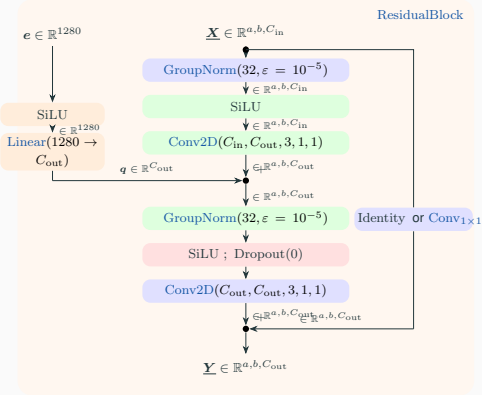

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# ResidualBlock

Input $\underline{\boldsymbol{X}} \in \mathbb{R}^{a,b,C_{\text{in}}}$, time $\boldsymbol{e} \in \mathbb{R}^{1280}$; output $\underline{\boldsymbol{Y}} \in \mathbb{R}^{a,b,C_{\text{out}}}$. All learned parameters are independent for the two convolution/normalization stages.

$$\begin{aligned}
\underline{\boldsymbol{U}} &= \text{Conv}_{3\times3}(\text{SiLU}(\text{GN}_{32}(\underline{\boldsymbol{X}}))),\\
\boldsymbol{q} &= \text{Linear}(\text{SiLU}(\boldsymbol{e})),\\
\underline{\boldsymbol{V}} &= \text{Conv}_{3\times3}(\text{SiLU}(\text{GN}_{32}(\underline{\boldsymbol{U}} + \boldsymbol{q}))),\\
\underline{\boldsymbol{Y}} &= \underline{\boldsymbol{V}} + \text{Shortcut}(\underline{\boldsymbol{X}}).
\end{aligned}$$

$\boldsymbol{q} \in \mathbb{R}^{C_{\text{out}}}$ is broadcast over $a, b$. Shortcut: identity if channels agree, otherwise $\text{Conv}_{1\times1}(C_{\text{in}} \to C_{\text{out}})$. $\varepsilon = 10^{-5}$, dropout $= 0$, output scale $= 1$.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Group normalization

Let $\underline{\boldsymbol{X}} \in \mathbb{R}^{a,b,C}$, $G \mid C$, and let $I_g$ contain the $C/G$ channels of group $g$. Define

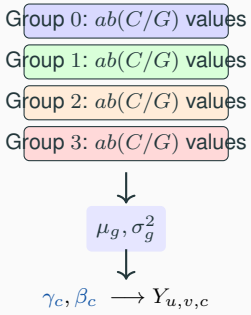
$$\mu_g = \frac{1}{ab(C/G)} \sum_{c \in I_g} \sum_{u,v} X_{u,v,c},$$

$$\sigma_g^2 = \frac{1}{ab(C/G)} \sum_{c \in I_g} \sum_{u,v} (X_{u,v,c} - \mu_g)^2.$$

For $c \in I_g$,

$$Y_{u,v,c} = \gamma_c \frac{X_{u,v,c} - \mu_g}{\sqrt{\sigma_g^2 + \varepsilon}} + \beta_c.$$

$\underline{\boldsymbol{Y}} \in \mathbb{R}^{a,b,C}$; $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^C$. In SD 1.5 ResNet: $G = 32$, $\varepsilon = 10^{-5}$; spatial Transformer normalization uses $\varepsilon = 10^{-6}$.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

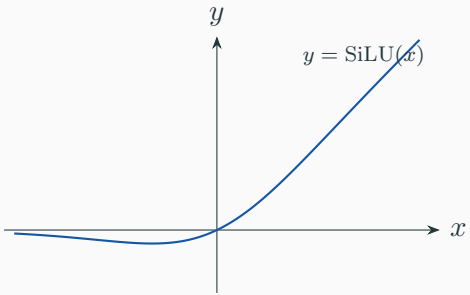
# SiLU activation

For a scalar $x \in \mathbb{R}$,

$$\mathrm{SiLU}(x) = \frac{x}{1+e^{-x}} = x\,\sigma(x), \quad \sigma(x) = \frac{1}{1+e^{-x}}.$$

For $\underline{\boldsymbol{X}} \in \mathbb{R}^{a,b,C}$, $Y_{u,v,c} = \mathrm{SiLU}(X_{u,v,c})$ and $\underline{\boldsymbol{Y}} \in \mathbb{R}^{a,b,C}$.

No learned parameters occur in SiLU itself.

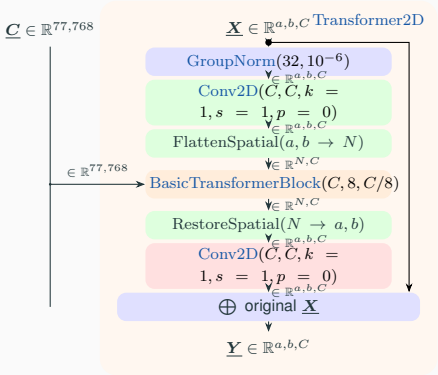



Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Transformer2D

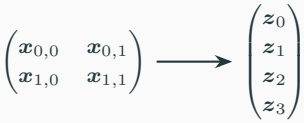
Input $\underline{\boldsymbol{X}} \in \mathbb{R}^{a,b,C}$; context $\underline{\boldsymbol{C}} \in \mathbb{R}^{77,768}$; output $\underline{\boldsymbol{Y}} \in \mathbb{R}^{a,b,C}$. $N = ab$.

The spatial normalization uses $32$ groups, $\varepsilon = 10^{-6}$. Both projections are learned convolutions with $k = 1, s = 1, p = 0$; one $\mathrm{BasicTransformerBlock}$ uses 8 heads of dimension $C/8$. Internal residual addition returns the original shape. For $C = 320, 640, 1280$, head dimensions equal $40, 80, 160$. No positional embedding is added by this module.



# Spatial flattening and restoring

Let $\underline{\boldsymbol{X}} \in \mathbb{R}^{a,b,C}$ and $N = ab$. For zero-based indices, flattening defines $\boldsymbol{z}_{uw+v} = \underline{\boldsymbol{X}}_{u,v,:} \in \mathbb{R}^C$, where $0 \leq u < a$, $0 \leq v < b$ and $w = b$. Then $\underline{\boldsymbol{Z}} \in \mathbb{R}^{N,C}$. Restoring uses the inverse index map $u = \lfloor n/b \rfloor$, $v = n \bmod b$; no learned parameters.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Spatial flattening — exercise

**Exercise**

*For $a = b = 2$ and $C = 3$, let $\underline{\boldsymbol{X}} = (\boldsymbol{X}_1, \boldsymbol{X}_2, \boldsymbol{X}_3)$, where*

$$\boldsymbol{X}_1 = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \boldsymbol{X}_2 = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}, \quad \boldsymbol{X}_3 = \begin{pmatrix} 9 & 10 \\ 11 & 12 \end{pmatrix}.$$

*Write the four token vectors $\boldsymbol{z}_0, \ldots, \boldsymbol{z}_3 \in \mathbb{R}^3$ in row-major order and the flattened matrix $\boldsymbol{Z} \in \mathbb{R}^{4,3}$. Then restore the three $2 \times 2$ channel matrices.*

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Spatial flattening — answer

**Answer**:

# Spatial flattening — answer

**Answer**: For $n = 2u + v$ (zero-based spatial indices), the channel values at each spatial location form

$$\boldsymbol{z}_0 = (1, 5, 9)^\top, \quad \boldsymbol{z}_1 = (2, 6, 10)^\top,$$
$$\boldsymbol{z}_2 = (3, 7, 11)^\top, \quad \boldsymbol{z}_3 = (4, 8, 12)^\top.$$

Thus

$$\boldsymbol{Z} = \begin{pmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{pmatrix} \in \mathbb{R}^{4,3}.$$

Restoring $u = \lfloor n/2 \rfloor$, $v = n \bmod 2$ gives

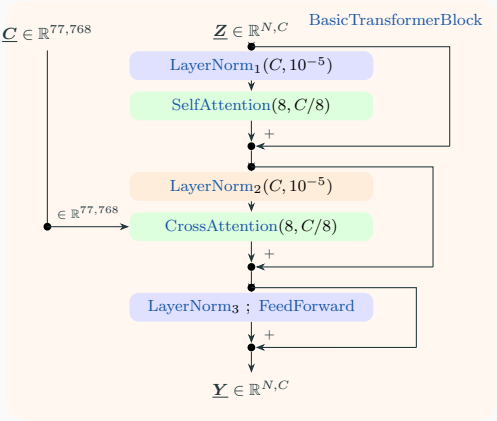
$$\boldsymbol{X}_1 = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \boldsymbol{X}_2 = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}, \quad \boldsymbol{X}_3 = \begin{pmatrix} 9 & 10 \\ 11 & 12 \end{pmatrix}.$$

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# BasicTransformerBlock

Input $\underline{\boldsymbol{Z}} \in \mathbb{R}^{N,C}$, text $\underline{\boldsymbol{C}} \in \mathbb{R}^{77,768}$. Eight heads; head width $C/8$, no causal mask. Three learned LayerNorms have $\varepsilon = 10^{-5}$.
Residual branch outputs are added after each self-attention, cross-attention, and feed-forward transformation:

$$\underline{\boldsymbol{Z}}_1 = \underline{\boldsymbol{Z}} + \mathrm{SelfAttention}(\mathrm{LN}_1(\underline{\boldsymbol{Z}})),$$
$$\underline{\boldsymbol{Z}}_2 = \underline{\boldsymbol{Z}}_1 + \mathrm{CrossAttention}(\mathrm{LN}_2(\underline{\boldsymbol{Z}}_1), \underline{\boldsymbol{C}}),$$
$$\underline{\boldsymbol{Y}} = \underline{\boldsymbol{Z}}_2 + \mathrm{FeedForward}(\mathrm{LN}_3(\underline{\boldsymbol{Z}}_2)).$$




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Layer normalization

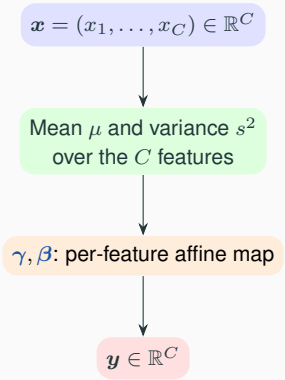
For token $\boldsymbol{x} \in \mathbb{R}^C$, put

$$\mu = \frac{1}{C}\sum_{c=1}^{C} x_c, \qquad s^2 = \frac{1}{C}\sum_{c=1}^{C}(x_c - \mu)^2.$$

Then $\boldsymbol{y} \in \mathbb{R}^C$ has components

$$y_c = \gamma_c \frac{x_c - \mu}{\sqrt{s^2 + \varepsilon}} + \beta_c.$$

For $N$ tokens the same $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^C$ are applied to each token separately. SD Transformer $\varepsilon = 10^{-5}$.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Self-attention: vectors and weights

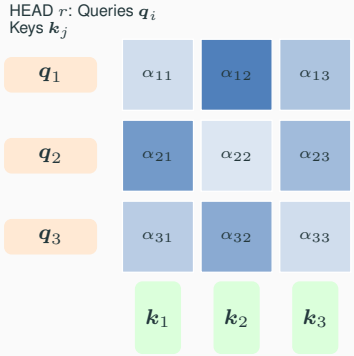
For $\boldsymbol{z}_i \in \mathbb{R}^C$, $1 \le i \le N$, and each head $r \in \{1, \dots, 8\}$, $d_h = C/8$, define

$$\boldsymbol{q}_i^{(r)} = \boldsymbol{W}_Q^{(r)} \boldsymbol{z}_i, \quad \boldsymbol{k}_j^{(r)} = \boldsymbol{W}_K^{(r)} \boldsymbol{z}_j, \quad \boldsymbol{v}_j^{(r)} = \boldsymbol{W}_V^{(r)} \boldsymbol{z}_j.$$

All projected vectors belong to $\mathbb{R}^{d_h}$; each matrix is in $\mathbb{R}^{d_h, C}$. Define scalars

$$s_{ij}^{(r)} = \frac{(\boldsymbol{q}_i^{(r)})^\top \boldsymbol{k}_j^{(r)}}{\sqrt{d_h}}, \quad \alpha_{ij}^{(r)} = \frac{\exp\left(s_{ij}^{(r)}\right)}{\sum_{\ell=1}^{N} \exp\left(s_{i\ell}^{(r)}\right)}.$$

Then $\sum_j \alpha_{ij}^{(r)} = 1$. No learnable parameters in softmax.
No causal mask in SD1.5 image self-attention.



Darker squares indicate larger weights

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Self-attention: weighted values and output

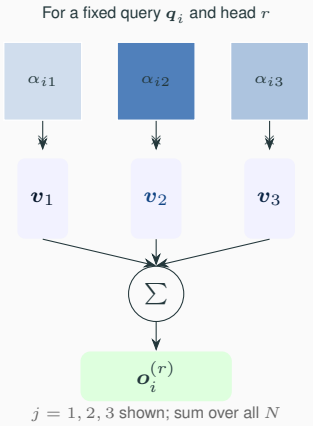
For each head $r$, define

$$\boldsymbol{o}_i^{(r)} = \sum_{j=1}^{N} \alpha_{ij}^{(r)} \boldsymbol{v}_j^{(r)} \in \mathbb{R}^{d_h}.$$

Concatenate the 8 head results:

$$\boldsymbol{o}_i = \begin{pmatrix} \boldsymbol{o}_i^{(1)} \\ \vdots \\ \boldsymbol{o}_i^{(8)} \end{pmatrix} \in \mathbb{R}^C, \quad \boldsymbol{y}_i = \boldsymbol{W}_O \boldsymbol{o}_i + \boldsymbol{b}_O \in \mathbb{R}^C.$$

$\boldsymbol{W}_O \in \mathbb{R}^{C,C}$; $\boldsymbol{b}_O \in \mathbb{R}^C$.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Self-attention weights — exercise

**Exercise**

*Consider one attention head with $d_h = 1$, two tokens, $\boldsymbol{q}_1 = (1)$, $\boldsymbol{k}_1 = (0)$ and $\boldsymbol{k}_2 = (\ln 3)$. Find the two weights $\alpha_{11}, \alpha_{12}$ exactly.*

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Self-attention weights — answer

**Answer**:

# Self-attention weights — answer

**Answer**: Because $\sqrt{d_h} = 1$, $s_{11} = 1 \cdot 0 = 0$ and $s_{12} = 1 \cdot \ln 3 = \ln 3$. Exponentials are $e^0 = 1$ and $e^{\ln 3} = 3$. Consequently

$$\alpha_{11} = \frac{1}{1+3} = \frac{1}{4}, \qquad \alpha_{12} = \frac{3}{1+3} = \frac{3}{4}.$$

The sum is $\frac{1}{4} + \frac{3}{4} = 1$.

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Self-attention weighted values — exercise

**Exercise**

*Use the weights* $\alpha_{11} = 1/4$, $\alpha_{12} = 3/4$, *values* $\boldsymbol{v}_1 = (2, -2)^\top$, $\boldsymbol{v}_2 = (0, 2)^\top \in \mathbb{R}^2$.
*Compute* $\boldsymbol{o}_1 = \alpha_{11}\boldsymbol{v}_1 + \alpha_{12}\boldsymbol{v}_2$.

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Self-attention weighted values — answer

**Answer**:

# Self-attention weighted values — answer

**Answer**:

$$\boldsymbol{o}_1 = \frac{1}{4}\begin{pmatrix} 2 \\ -2 \end{pmatrix} + \frac{3}{4}\begin{pmatrix} 0 \\ 2 \end{pmatrix} = \begin{pmatrix} 1/2 \\ -1/2 \end{pmatrix} + \begin{pmatrix} 0 \\ 3/2 \end{pmatrix} = \begin{pmatrix} 1/2 \\ 1 \end{pmatrix} \in \mathbb{R}^2.$$

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

## Pros and cons of convolutional layer

The self-attention layer can gather global information at once.

## Pros and cons of convolutional layer

The self-attention layer can gather global information at once.

One concern is the computational complexity. It is $O((hw)^2)$, which is acceptable.

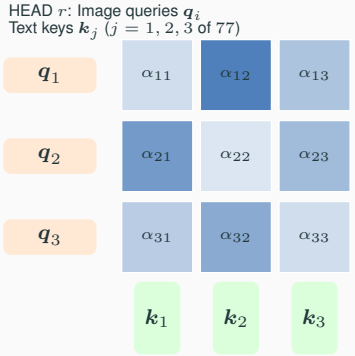
# Cross-attention: text keys and values

Image-token input: $\boldsymbol{z}_i \in \mathbb{R}^C$, $i = 1, \ldots, N$. Text tokens: $\boldsymbol{c}_j \in \mathbb{R}^{768}$, $j = 1, \ldots, 77$. For head $r$ with $d_h = C/8$,

$$\boldsymbol{q}_i^{(r)} = \boldsymbol{W}_Q^{(r)} \boldsymbol{z}_i, \quad \boldsymbol{k}_j^{(r)} = \boldsymbol{W}_K^{(r)} \boldsymbol{c}_j, \quad \boldsymbol{v}_j^{(r)} = \boldsymbol{W}_V^{(r)} \boldsymbol{c}_j.$$

$\boldsymbol{W}_Q^{(r)} \in \mathbb{R}^{d_h, C}$; $\boldsymbol{W}_K^{(r)}, \boldsymbol{W}_V^{(r)} \in \mathbb{R}^{d_h, 768}$; all projected vectors in $\mathbb{R}^{d_h}$.

$$\alpha_{ij}^{(r)} = \frac{\exp\left( (\boldsymbol{q}_i^{(r)})^\top \boldsymbol{k}_j^{(r)} / \sqrt{d_h} \right)}{\sum_{\ell=1}^{77} \exp\left( (\boldsymbol{q}_i^{(r)})^\top \boldsymbol{k}_\ell^{(r)} / \sqrt{d_h} \right)}.$$




First three text positions shown

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Cross-attention: weighted text vectors

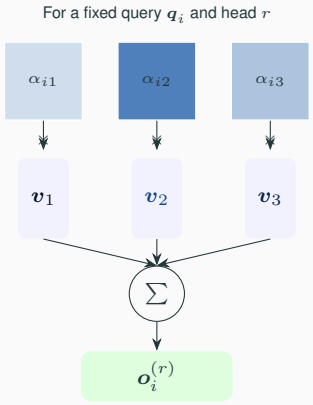
For one image query token $i$ and head $r$,

$$\boldsymbol{o}_i^{(r)} = \sum_{j=1}^{77} \alpha_{ij}^{(r)} \boldsymbol{v}_j^{(r)} \in \mathbb{R}^{d_h}.$$

Concatenate all eight heads and project:

$$\boldsymbol{o}_i = \begin{pmatrix} \boldsymbol{o}_i^{(1)} \\ \vdots \\ \boldsymbol{o}_i^{(8)} \end{pmatrix} \in \mathbb{R}^C, \quad \boldsymbol{y}_i = \boldsymbol{W}_O \boldsymbol{o}_i + \boldsymbol{b}_O.$$

Then $\boldsymbol{y}_i \in \mathbb{R}^C$. The output has $N$ image tokens even though attention is taken over 77 text tokens.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Cross-attention weights — exercise

**Exercise**

*Use one head with $d_h = 3$ and two text tokens. For the image query*

$$\boldsymbol{q} = (\sqrt{3}, 0, 0)^\top \in \mathbb{R}^3$$

*and the text keys*

$$\boldsymbol{k}_1 = (1, 0, 0)^\top, \qquad \boldsymbol{k}_2 = (0, 1, 0)^\top \in \mathbb{R}^3,$$

*calculate the scaled dot-product scores and the two cross-attention weights. Approximate the weights to four decimal places.*

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Cross-attention weights — answer

**Answer**:

# Cross-attention weights — answer

**Answer**: The two dot products and scaled scores are

$$\boldsymbol{q}^\top \boldsymbol{k}_1 = \sqrt{3}, \quad s_1 = \frac{\sqrt{3}}{\sqrt{d_h}} = 1,$$
$$\boldsymbol{q}^\top \boldsymbol{k}_2 = 0, \qquad s_2 = \frac{0}{\sqrt{3}} = 0.$$

Consequently, the two attention weights are

$$\alpha_1 = \frac{e^1}{e^1 + e^0} = \frac{e}{e+1} \approx 0.7311, \qquad \alpha_2 = \frac{e^0}{e^1 + e^0} = \frac{1}{e+1} \approx 0.2689.$$

They sum to $1$.

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Cross-attention weighted values — exercise

**Exercise**

*For the cross-attention weights $\alpha_1 = 2/3$, $\alpha_2 = 1/3$, let text values be $\boldsymbol{v}_1 = (3, 0)^\top$, $\boldsymbol{v}_2 = (0, 6)^\top$. Calculate the head output $\boldsymbol{o} = \alpha_1 \boldsymbol{v}_1 + \alpha_2 \boldsymbol{v}_2$.*

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Cross-attention weighted values — answer

**Answer**:

# Cross-attention weighted values — answer

**Answer**:

$$\boldsymbol{o} = \frac{2}{3}\begin{pmatrix}3\\0\end{pmatrix} + \frac{1}{3}\begin{pmatrix}0\\6\end{pmatrix} = \begin{pmatrix}2\\0\end{pmatrix} + \begin{pmatrix}0\\2\end{pmatrix} = \begin{pmatrix}2\\2\end{pmatrix} \in \mathbb{R}^2.$$

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

## Cross-attention Pros and cons

The cross attention layer can bring the user input information. However, each pixel location cannot gather the information from other pixel.

## Cross-attention Pros and cons

The cross attention layer can bring the user input information. However, each pixel location cannot gather the information from other pixel.

The computational complexity. It is $O(M(hw))$, which is acceptable.

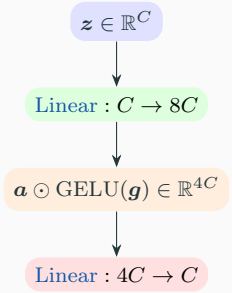
# FeedForward and GEGLU

For $\boldsymbol{z} \in \mathbb{R}^C$, $\mathrm{FeedForward}$ in SD 1.5 uses GEGLU with hidden size $D = 4C$:

$$\begin{pmatrix} \boldsymbol{a} \\ \boldsymbol{g} \end{pmatrix} = \boldsymbol{A}\boldsymbol{z} + \boldsymbol{b} \in \mathbb{R}^{2D}, \qquad \boldsymbol{u} = \boldsymbol{a} \odot \mathrm{GELU}(\boldsymbol{g}) \in \mathbb{R}^D,$$

$$\boldsymbol{y} = \boldsymbol{B}\boldsymbol{u} + \boldsymbol{c} \in \mathbb{R}^C.$$

$\boldsymbol{A} \in \mathbb{R}^{2D,C}$, $\boldsymbol{B} \in \mathbb{R}^{C,D}$, dropout $0$. For $N$ tokens, same operations are applied to each token.
$\mathrm{GELU}(x) = x\Phi(x)$, where
$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} \exp\left(-u^2/2\right) du$.



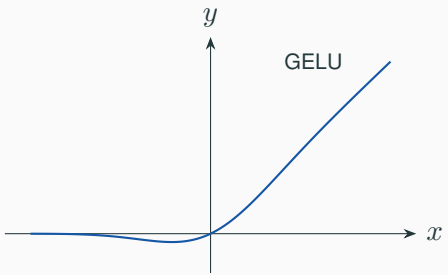

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# GELU activation

The exact Gaussian error linear unit is

$$\mathrm{GELU}(x) = x\Phi(x), \quad \Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-u^2/2}\, du.$$

For $\boldsymbol{g} \in \mathbb{R}^D$, GELU is applied to each component. $\mathrm{GELU}$ has no learned parameters. In GEGLU, its result is multiplied componentwise by another learned projection.



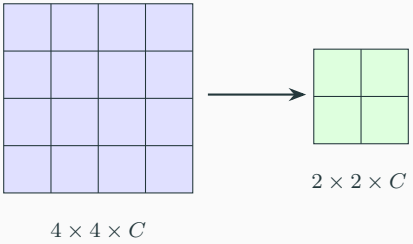

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Strided Downsample2D

$\mathrm{Downsample2D}$ in the first three down blocks is one learned 2D convolution:

$$\underline{\boldsymbol{X}} \in \mathbb{R}^{a,b,C} \xrightarrow{\mathrm{Conv2D}(C,C,3,2,1)} \underline{\boldsymbol{Y}} \in \mathbb{R}^{a/2,b/2,C}.$$

For even $a, b$, the formula $a' = \lfloor (a + 2 - 3)/2 \rfloor + 1 = a/2$ holds; similarly for $b$. This is not average pooling.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Strided Downsample2D — exercise

**Exercise**

*For a single-channel $4 \times 4$ input, a $3 \times 3$ all-one kernel, $s = 2$, $p = 1$, and zero bias, calculate the output shape and its top-left element when $\boldsymbol{X}$ contains all ones.*

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Strided Downsample2D — answer

**Answer**:

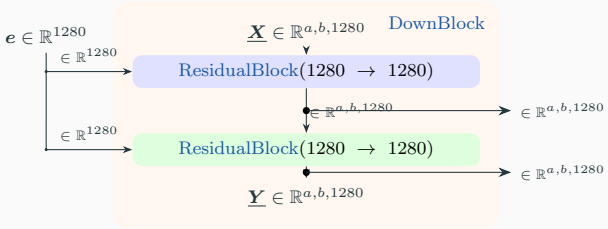
# Strided Downsample2D — answer

**Answer**: Each spatial dimension is $\lfloor (4 + 2 - 3)/2 \rfloor + 1 = 2$. The top-left $3 \times 3$ window overlaps the input in its lower-right $2 \times 2$ portion, containing four ones; the other five positions are zero padding. Thus $Y_{0,0} = 4$ and $\underline{\boldsymbol{Y}} \in \mathbb{R}^{2,2,1}$.

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# DownBlock

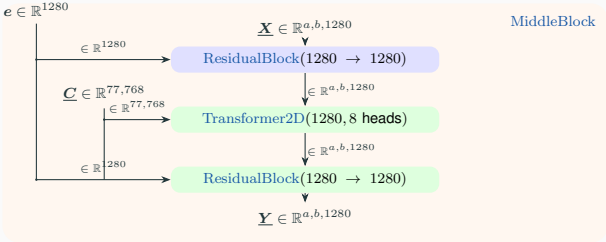
The fourth down block has two residual blocks and no downsampling or attention. All spatial dimensions remain $a = b = h/8$ for square $h = w$; more generally $a = h/8, b = w/8$.
$C = 1280$, time $\boldsymbol{e} \in \mathbb{R}^{1280}$. Both residual outputs are saved separately, to be consumed by the first up block.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# MiddleBlock

The middle U-Net block preserves $a = h/8, b = w/8, C = 1280$. It contains a ResidualBlock, one Transformer2D module, and another ResidualBlock. The Transformer2D contains exactly one BasicTransformerBlock.
Both ResNet operations accept $\boldsymbol{e} \in \mathbb{R}^{1280}$. The Transformer2D accepts $\underline{\boldsymbol{C}} \in \mathbb{R}^{77,768}$.

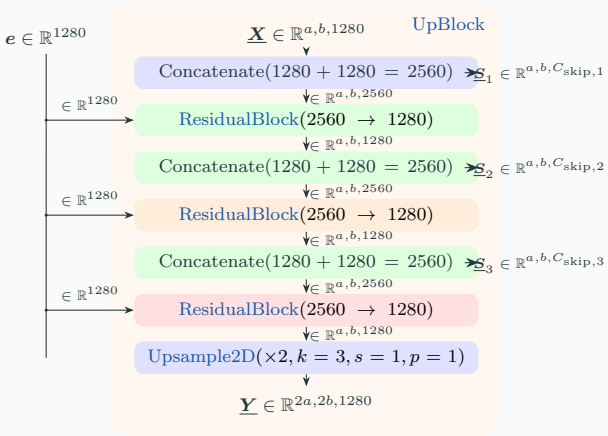



Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# UpBlock

The first up block uses $\underline{\boldsymbol{S}}_1, \underline{\boldsymbol{S}}_2, \underline{\boldsymbol{S}}_3 \in \mathbb{R}^{a,b,1280}$, consumed in reverse order: $\underline{\boldsymbol{S}}_1$ and $\underline{\boldsymbol{S}}_2$ are the second and first ResidualBlock outputs of $\mathrm{DownBlock}_4$, while $\underline{\boldsymbol{S}}_3 = \underline{\boldsymbol{S}}_{3,3}$ comes from $\mathrm{CrossAttentionDownBlock}_3$. Here $a = h/8$, $b = w/8$. For each $r = 1, 2, 3$, the corresponding concatenation is $\underline{\boldsymbol{U}}_r = \mathrm{ConcatChannels}(\underline{\boldsymbol{X}}_r, \underline{\boldsymbol{S}}_r) \in \mathbb{R}^{a,b,2560}$, followed by a learned ResidualBlock.

Each concatenation increases channels from 1280 to 2560. A learned ResidualBlock reduces channels from 2560 to 1280. Nearest-neighbor resizing followed by learned $3 \times 3$



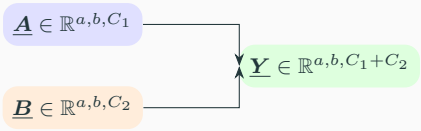
# Channel concatenation

For $\underline{\boldsymbol{A}} \in \mathbb{R}^{a,b,C_1}$ and $\underline{\boldsymbol{B}} \in \mathbb{R}^{a,b,C_2}$, define

$$\mathrm{Concatenate}(\underline{\boldsymbol{A}}, \underline{\boldsymbol{B}}) = \underline{\boldsymbol{Y}} \in \mathbb{R}^{a,b,C_1+C_2},$$

$$Y_{u,v,c} = \begin{cases} A_{u,v,c}, & 1 \le c \le C_1, \\ B_{u,v,c-C_1}, & C_1 < c \le C_1 + C_2. \end{cases}$$

This is channel concatenation, not addition.
It has no learned parameters.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Upsample2D: nearest-neighbor and convolution

Let $\underline{\boldsymbol{X}} \in \mathbb{R}^{a,b,C}$. Nearest-neighbor resizing by factor 2 defines

$$U_{u,v,c} = X_{\lceil u/2 \rceil, \lceil v/2 \rceil, c} \quad (1 \le u \le 2a, 1 \le v \le 2b).$$

Then $\underline{\boldsymbol{U}} \in \mathbb{R}^{2a,2b,C}$. SD 1.5 next applies $\mathrm{Conv2D}(C, C, k = 3, s = 1, p = 1)$ to obtain $\underline{\boldsymbol{Y}} \in \mathbb{R}^{2a,2b,C}$. The resize has no parameters, but the convolution is learned.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \\ 3 & 3 & 4 & 4 \\ 3 & 3 & 4 & 4 \end{pmatrix}$$

$2 \times 2$ $\qquad$ $4 \times 4$

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Upsample2D — exercise

**Exercise**

*For* $\boldsymbol{X} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \in \mathbb{R}^{2,2}$*, calculate the nearest-neighbor upsampling result with scale factor 2.*

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Upsample2D — answer

**Answer**:

# Upsample2D — answer

**Answer**: Nearest-neighbor resizing replicates each input element in a $2 \times 2$ square:

$$\boldsymbol{U} = \begin{pmatrix} 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \\ 3 & 3 & 4 & 4 \\ 3 & 3 & 4 & 4 \end{pmatrix}.$$

A center-only convolution with $s = 1, p = 1$ copies each entry; therefore $\boldsymbol{Y} = \boldsymbol{U} \in \mathbb{R}^{4,4}$.

Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# CrossAttentionUpBlock

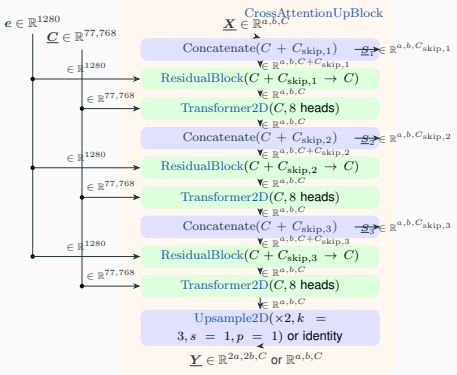
This block repeats concatenate skip → ResidualBlock → Transformer2D three times; it then upsamples except in the last (fourth) U-Net up block.
For Up blocks 2,3,4, $(C, C_{\text{skip},1}, C_{\text{skip},2}, C_{\text{skip},3})$ are respectively

$$(1280, 1280, 1280, 640),$$

$$(640, 640, 640, 320),$$

$$(320, 320, 320, 320).$$

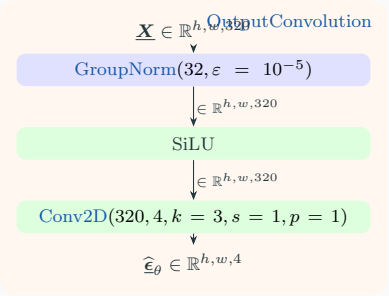
The three input skips $(\underline{\boldsymbol{S}}_1, \underline{\boldsymbol{S}}_2, \underline{\boldsymbol{S}}_3)$ are, respectively, $(\underline{\boldsymbol{S}}_{3,2}, \underline{\boldsymbol{S}}_{3,1}, \underline{\boldsymbol{S}}_{2,3})$ in Up 2, $(\underline{\boldsymbol{S}}_{2,2}, \underline{\boldsymbol{S}}_{2,1}, \underline{\boldsymbol{S}}_{1,3})$ in Up 3, and $(\underline{\boldsymbol{S}}_{1,2}, \underline{\boldsymbol{S}}_{1,1}, \underline{\boldsymbol{X}}_{\text{input}})$ in Up 4, where $\underline{\boldsymbol{X}}_{\text{input}} \in \mathbb{R}^{h,w,320}$ is the InputConvolution output. Each right-hand skip enters $\underline{\boldsymbol{U}}_r = \text{ConcatChannels}(\underline{\boldsymbol{X}}_r, \underline{\boldsymbol{S}}_r) \in \mathbb{R}^{a,b,C+C_{\text{skip},r}}$ before its ResidualBlock. The first/second/third residual input channels are $(2560, 2560, 1920)$, $(1920, 1280, 960)$, $(960, 640, 640)$; all output $C$ channels. Head count $8$, text width 768; time width 1280.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# OutputConvolution

The final U-Net operations preserve spatial shape and map 320 channels to 4. The GroupNorm uses $32$ groups, $\varepsilon = 10^{-5}$, learned affine parameters; the output convolution uses kernel $3 \times 3$, stride 1, padding 1 and learned bias.
$\underline{\boldsymbol{X}} \in \mathbb{R}^{h,w,320}$; $\underline{\widehat{\boldsymbol{\epsilon}}}_\theta \in \mathbb{R}^{h,w,4}$. No text or time signal is directly supplied to this final block.




Source: Diffusers v0.35.2: UNet2DConditionModel, `unet_2d_blocks.py`, `resnet.py`, `transformer_2d.py`, `attention.py`.

# Outline

1. Introduction

2. Mathematical notation

3. What are the necessary conditions for NNs in iterative generation?

**4. How does Stable Diffusion 1.5's U-Net work?**

5. Diffusion Transformer of Qwen-Image-2.1

6. Summary

# Outline

1. Introduction

2. Mathematical notation

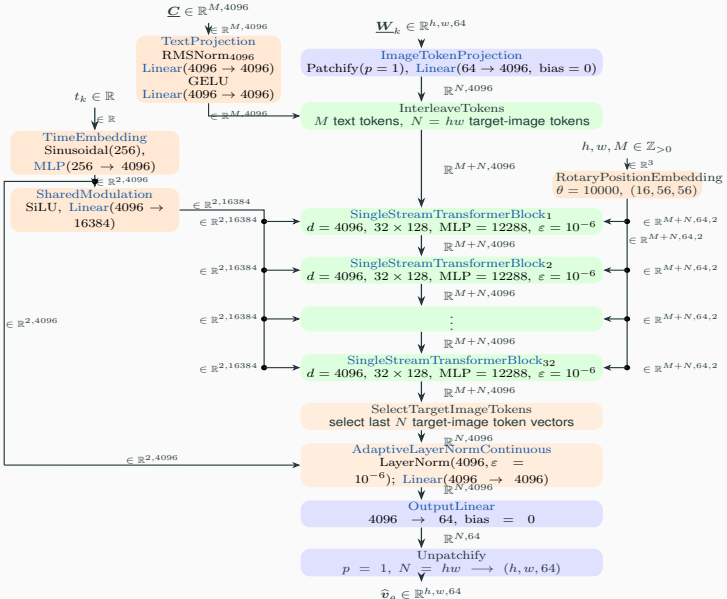
3. What are the necessary conditions for NNs in iterative generation?

4. How does Stable Diffusion 1.5's U-Net work?

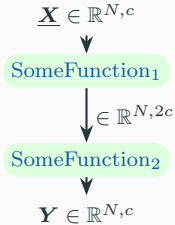
**5. Diffusion Transformer of Qwen-Image-2.1**

6. Summary

# Qwen-Image-2.1 DiT: the whole structure




## Legend




Colored boxes: functions. Arrows: tensors; annotations: tensor shapes.

$h = H/16$, $w = W/16$; $N = hw$; $M$: text-token count.

RoPE and shared modulation enter all 32 Transformer blocks.

With `causal_condition`, text uses $t = 0$, target image uses $t_k$.

DiT predicts $\widehat{\underline{\boldsymbol{v}}}_\theta$, not $\underline{\boldsymbol{W}}_{k+1}$.

Source: Qwen-Image-2.1 transformer config; Diffusers QwenImage21Transformer2DModel.

## Which parts should we focus?

Focus on where we integrate the current latent image's information $\underline{\boldsymbol{W}}_k$ and the user input information $\underline{\boldsymbol{C}}$.

## Which parts should we focus?

Focus on where we integrate the current latent image's information $\underline{\boldsymbol{W}_k}$ and the user input information $\underline{\boldsymbol{C}}$.

**Exercise**

*Where?*

# Which parts should we focus?

**Answer**:

## Which parts should we focus?

**Answer**: In Qwen-Image-2.1, we first concatenate both pieces of information in InterleaveTokens, and integrate them in SingleStreamTransformerBlock.

## Which parts should we focus?

**Answer**: In Qwen-Image-2.1, we first concatenate both pieces of information in $\mathrm{InterleaveTokens}$, and integrate them in $\mathrm{SingleStreamTransformerBlock}$.

The $\mathrm{SingleStreamTransformerBlock}$ also receives the information of the time. Moreover, since the Transformer itself cannot capture the positional information, we input the rotary position embedding (RoPE) to $\mathrm{SingleStreamTransformerBlock}$.

## Which parts should we focus?

**Answer**: In Qwen-Image-2.1, we first concatenate both pieces of information in InterleaveTokens, and integrate them in SingleStreamTransformerBlock.

The SingleStreamTransformerBlock also receives the information of the time. Moreover, since the Transformer itself cannot capture the positional information, we input the rotary position embedding (RoPE) to SingleStreamTransformerBlock.

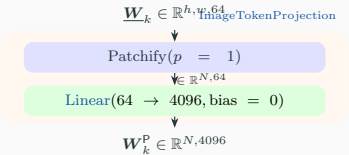
Since InterleaveTokens enlarges the shape of the tensor, SelectTargetImageTokens removes the redundant parts.

# ImageTokenProjection

$h = H/16$, $w = W/16$, $N = hw$. The VAE latent input $\underline{\boldsymbol{W}}_k \in \mathbb{R}^{h,w,64}$ is unrolled into $N$ vectors $\boldsymbol{w}_j \in \mathbb{R}^{64}$. Since $p = 1$, each patch is one pixel in the latent grid.

$$\boldsymbol{w}_j^{\mathsf{P}} = \boldsymbol{A}_{\text{img}} \boldsymbol{w}_j \in \mathbb{R}^{4096}, \quad \boldsymbol{A}_{\text{img}} \in \mathbb{R}^{4096,64}.$$

The projection has no bias. $\boldsymbol{W}_k^{\mathsf{P}} \in \mathbb{R}^{N,4096}$.



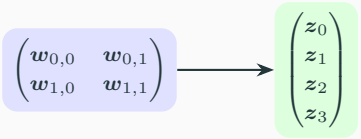

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Patchify with patch size one

Let $\underline{\boldsymbol{W}} \in \mathbb{R}^{h,w,64}$. With $p = 1$, each latent pixel is one patch:

$$\boldsymbol{z}_{uw+v} = \underline{\boldsymbol{W}}_{u,v,:} \in \mathbb{R}^{64}, \quad 0 \le u < h,\ 0 \le v < w.$$

The flattened tokens form $\underline{\boldsymbol{Z}} \in \mathbb{R}^{hw,64}$. No values are changed and no parameters are learned.




$\mathbb{R}^{2,2,64} \to \mathbb{R}^{4,64}$

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Patchify with patch size one — exercise

**Exercise**

*For $h = w = 2$, $C = 3$ and $p = 1$, let $\underline{\boldsymbol{W}} = (\boldsymbol{W}_1, \boldsymbol{W}_2, \boldsymbol{W}_3) \in \mathbb{R}^{2,2,3}$, where*

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \boldsymbol{W}_2 = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}, \quad \boldsymbol{W}_3 = \begin{pmatrix} 9 & 10 \\ 11 & 12 \end{pmatrix}.$$

*List the four patch vectors in row-major order and give the shape of the resulting token matrix.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Patchify with patch size one — answer

**Answer**:

# Patchify with patch size one — answer

**Answer**: Each pixel contributes its three channel values:

$$\boldsymbol{z}_0 = \begin{pmatrix} 1 \\ 5 \\ 9 \end{pmatrix}, \quad \boldsymbol{z}_1 = \begin{pmatrix} 2 \\ 6 \\ 10 \end{pmatrix}, \quad \boldsymbol{z}_2 = \begin{pmatrix} 3 \\ 7 \\ 11 \end{pmatrix}, \quad \boldsymbol{z}_3 = \begin{pmatrix} 4 \\ 8 \\ 12 \end{pmatrix}.$$

Hence the row-major token matrix is

$$\boldsymbol{Z} = \begin{pmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{pmatrix} \in \mathbb{R}^{4,3}.$$

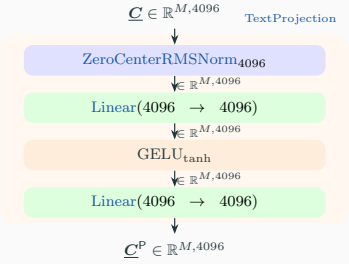
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# TextProjection

Input: $\underline{\boldsymbol{C}} \in \mathbb{R}^{M,4096}$, each $\boldsymbol{c}_i \in \mathbb{R}^{4096}$.

$$
\begin{aligned}
\boldsymbol{r}_i &= \mathrm{ZeroCenterRMSNorm}(\boldsymbol{c}_i), \\
\boldsymbol{a}_i &= \boldsymbol{A}\boldsymbol{r}_i, \\
\boldsymbol{g}_i &= \mathrm{GELU}(\boldsymbol{a}_i), \\
\boldsymbol{c}_i^{\mathsf{P}} &= \boldsymbol{B}\boldsymbol{g}_i.
\end{aligned}
$$

$\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{4096,4096}$, both without bias.
Output $\underline{\boldsymbol{C}}^{\mathsf{P}} \in \mathbb{R}^{M,4096}$.



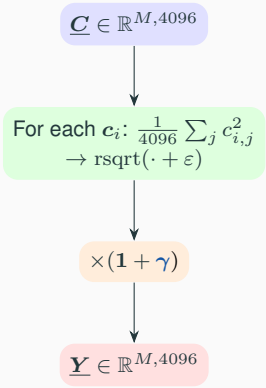

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# ZeroCenterRMSNorm

Input $\underline{\boldsymbol{C}} \in \mathbb{R}^{M,4096}$; for each token $\boldsymbol{c}_i \in \mathbb{R}^{4096}$, the learned weights are stored zero-centered:

$$\boldsymbol{y}_i = \frac{\boldsymbol{c}_i}{\sqrt{\frac{1}{4096}\sum_{j=1}^{4096} c_{i,j}^2 + 10^{-6}}} \odot (\mathbf{1} + \boldsymbol{\gamma}) \in \mathbb{R}^{4096},$$

where $\boldsymbol{\gamma} \in \mathbb{R}^{4096}$. Collecting the output vectors gives $\underline{\boldsymbol{Y}} \in \mathbb{R}^{M,4096}$. No mean subtraction or learned bias. Computation uses fp32 internally.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# ZeroCenterRMSNorm — exercise

**Exercise**

*Let three 2-dimensional token vectors form the input matrix*

$$\boldsymbol{C} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & -2 \end{pmatrix} \in \mathbb{R}^{3,2}.$$

*With $\varepsilon = 0$ and nonzero learned weights $\boldsymbol{\gamma} = (1, -\frac{1}{2})^\top \in \mathbb{R}^2$, apply ZeroCenterRMSNorm to each row and find $\boldsymbol{Y} \in \mathbb{R}^{3,2}$.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# ZeroCenterRMSNorm — answer

**Answer**:

# ZeroCenterRMSNorm — answer

**Answer**: The multiplicative scale is $\mathbf{1} + \boldsymbol{\gamma} = (2, \frac{1}{2})^\top$. The three row-wise root-mean-squares are

$$\sqrt{\frac{1^2 + 1^2}{2}} = 1, \qquad \sqrt{\frac{2^2 + 0^2}{2}} = \sqrt{2}, \qquad \sqrt{\frac{0^2 + (-2)^2}{2}} = \sqrt{2}.$$

Consequently,

$$\boldsymbol{y}_1 = \begin{pmatrix} 2 \\ 1/2 \end{pmatrix}, \quad \boldsymbol{y}_2 = \begin{pmatrix} 2\sqrt{2} \\ 0 \end{pmatrix}, \quad \boldsymbol{y}_3 = \begin{pmatrix} 0 \\ -1/\sqrt{2} \end{pmatrix},$$

and

$$\boldsymbol{Y} = \begin{pmatrix} 2 & 1/2 \\ 2\sqrt{2} & 0 \\ 0 & -1/\sqrt{2} \end{pmatrix} \in \mathbb{R}^{3,2}.$$

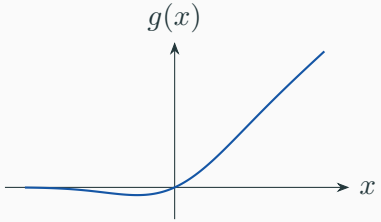
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# GELU with tanh approximation

QwenImage21TextProjection uses the tanh-approximate GELU:

$$g(x) = \frac{x}{2}\left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right]\right).$$

For $\boldsymbol{x} \in \mathbb{R}^{4096}$ the function is applied componentwise. No learned parameters occur in GELU itself.

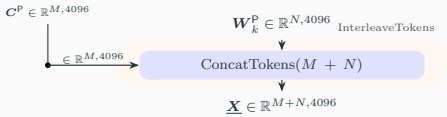



Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# InterleaveTokens

For text-to-image with no condition images, input projected text $\boldsymbol{C}^{\mathsf{P}} \in \mathbb{R}^{M,4096}$ and projected image $\boldsymbol{W}_k^{\mathsf{P}} \in \mathbb{R}^{N,4096}$. The text prefix is followed by the target image:

$$\underline{\boldsymbol{X}} = \mathrm{ConcatTokens}(\boldsymbol{C}^{\mathsf{P}}, \boldsymbol{W}_k^{\mathsf{P}}) \in \mathbb{R}^{M+N,4096}.$$




Writing the rows as $\boldsymbol{c}_i^{\mathsf{P}}, \boldsymbol{w}_j^{\mathsf{P}} \in \mathbb{R}^{4096}$, the concatenation is

$$\boldsymbol{x}_i = \begin{cases} \boldsymbol{c}_i^{\mathsf{P}}, & 1 \le i \le M, \\ \boldsymbol{w}_{i-M}^{\mathsf{P}}, & M < i \le M+N. \end{cases}$$

No learned parameters. In image-conditioned generation, image latent tokens instead replace the corresponding vision-language image slots in the joint prefix.

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Concatenate token sequences — exercise

**Exercise**

*Let* $M = 3, N = 2, d = 3$ *with projected text and image tokens*

$$\boldsymbol{C}^{\mathsf{P}} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \in \mathbb{R}^{3,3}, \qquad \boldsymbol{W}_k^{\mathsf{P}} = \begin{pmatrix} 10 & 11 & 12 \\ 13 & 14 & 15 \end{pmatrix} \in \mathbb{R}^{2,3}.$$

*Find* $\underline{\boldsymbol{X}} = \mathrm{ConcatTokens}(\boldsymbol{C}^{\mathsf{P}}, \boldsymbol{W}_k^{\mathsf{P}})$ *and its shape.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Concatenate token sequences — answer

**Answer**:

# Concatenate token sequences — answer

**Answer**: The text rows come first, followed by the two image rows:

$$\underline{\boldsymbol{X}} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \\ 13 & 14 & 15 \end{pmatrix} \in \mathbb{R}^{5,3}.$$

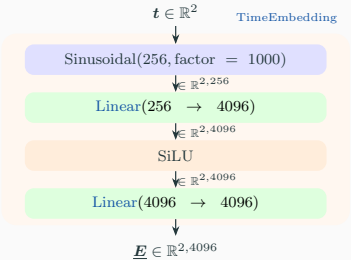
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# QwenImage21 TimeEmbedding

For $\text{causal_condition} = \texttt{True}$ and one sample, use $\boldsymbol{t} = (t_k, 0)^\top \in \mathbb{R}^2$. The sinusoidal function has dimension 256 and time factor 1000. The learned TimestepEmbedding MLP uses $256 \to 4096 \to 4096$ with SiLU.

$$\underline{\boldsymbol{E}} = \text{TimeEmbedding}(\boldsymbol{t}) \in \mathbb{R}^{2,4096}.$$

Row 1 conditions the target-image tokens and row 2 conditions prefix tokens.

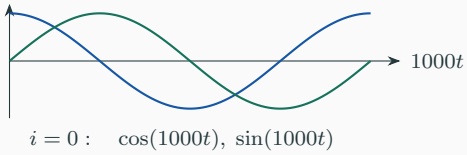



Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# QwenImage21 sinusoidal timesteps

For $t \in \mathbb{R}$, $d = 256$, and $0 \le i < 128$,

$$e_i(t) = \cos\Big(1000t \cdot 10000^{-i/128}\Big),$$
$$e_{128+i}(t) = \sin\Big(1000t \cdot 10000^{-i/128}\Big).$$




$\boldsymbol{e}(t) \in \mathbb{R}^{256}$, no learnable parameters. Relative to the SD1.5 embedding, Qwen uses $1000t$ rather than $t$ and $d = 256$.

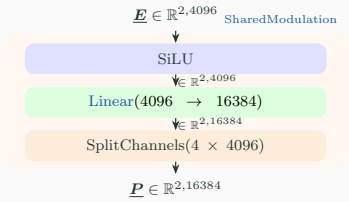
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# SharedModulation

Input $\underline{\boldsymbol{E}} \in \mathbb{R}^{2,4096}$. Apply SiLU and a single learned bias-free linear map, shared by all 32 transformer blocks:

$$\underline{\boldsymbol{P}} = \mathrm{Linear}(\mathrm{SiLU}(\underline{\boldsymbol{E}})) \in \mathbb{R}^{2,16384}.$$

Slice channels into 4 vectors of width 4096: $\boldsymbol{s}_1, \boldsymbol{g}_1, \boldsymbol{s}_2, \boldsymbol{g}_2$. Each token chooses the time-conditioned row according to its target-token mask.



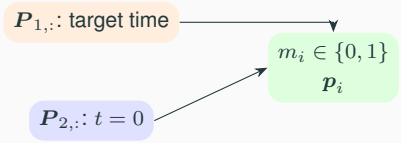

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Select modulation rows

In a single-sample forward, let $\underline{\boldsymbol{P}} \in \mathbb{R}^{2,16384}$ and a Boolean target-token mask $m_i \in \{0, 1\}$. For $1 \leq i \leq L$, define

$$\boldsymbol{p}_i = \begin{cases} \boldsymbol{P}_{1,:}, & m_i = 1, \\ \boldsymbol{P}_{2,:}, & m_i = 0. \end{cases} \in \mathbb{R}^{16384}.$$

$\underline{\boldsymbol{P}}_{\text{token}} \in \mathbb{R}^{L,16384}$. The selection is deterministic, not learned. The second row corresponds to timestep $0$.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Select modulation rows — exercise

**Exercise**

*Let $\boldsymbol{P}_{1,:} = (1, 2)^\top$, $\boldsymbol{P}_{2,:} = (3, 4)^\top$ and target mask $(0, 1, 1)$. Write the token-wise modulation matrix.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Select modulation rows — answer

**Answer**:

# Select modulation rows — answer

**Answer**: $\underline{\boldsymbol{P}}_{\text{token}} = \begin{pmatrix} 3 & 4 \\ 1 & 2 \\ 1 & 2 \end{pmatrix} \in \mathbb{R}^{3,2}$.

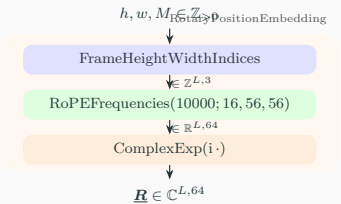
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# RotaryPositionEmbedding

Independent positional metadata for text-to-image: $h, w, M \in \mathbb{Z}_{>0}$ and $L = M + hw$. Assign 3-axis indices to every text or image token. Frame/height/width rotary dimensions are $(16, 56, 56)$; $16 + 56 + 56 = 128$. The rotary frequency table has $64$ complex components per token.

$$\underline{\boldsymbol{R}} \in \mathbb{C}^{L,64} \cong \mathbb{R}^{L,64,2}.$$

No learned parameters. The same $\underline{\boldsymbol{R}}$ modifies keys and queries in all 32 blocks.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# RotaryPositionEmbedding: 3-axis frequencies

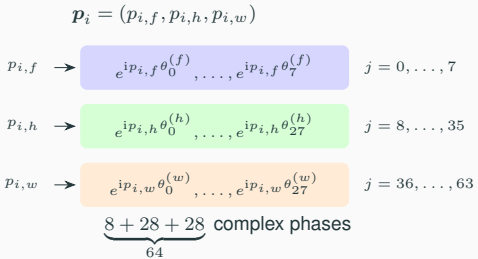
Let the $i$-th token have a three-axis position $\boldsymbol{p}_i = (p_{i,f}, p_{i,h}, p_{i,w}) \in \mathbb{Z}^3$. The complete position table is $\boldsymbol{P} \in \mathbb{Z}^{L,3}$, where $L = M + hw$.
Text token $i = 0, \ldots, M-1$ has $\boldsymbol{p}_i = (i, i, i)$. Image token $i = M + uw + v$ at zero-based row $u$ and column $v$ has

$$\boldsymbol{p}_{M+uw+v} = (M, u - \lceil h/2 \rceil, v - \lceil w/2 \rceil).$$

For axis $a \in \{f, h, w\}$, use $p = p_{i,a}$, $d_a \in \{16, 56, 56\}$, and $0 \le j < d_a/2$:

$$R^{(a)}_{p,j} = \cos\left(p\theta^{(a)}_j\right) + \mathrm{i}\sin\left(p\theta^{(a)}_j\right), \quad \theta^{(a)}_j = 10000^{-2j/d_a}.$$

Concatenating the $8 + 28 + 28$ phases gives $\boldsymbol{R}_i \in \mathbb{C}^{64}$ and $\underline{\boldsymbol{R}} \in \mathbb{C}^{L,64}$.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# RotaryPositionEmbedding: phases — exercise

**Exercise**

*Consider a toy 3-axis RoPE with $d_f = d_h = d_w = 2$ and base $10000$. For text-to-image with $M = 1$ and $h = w = 2$, consider the target-image token at zero-based position $(u, v) = (1, 0)$.*

*Find its three-axis position $\boldsymbol{p}_i = (p_{i,f}, p_{i,h}, p_{i,w}) \in \mathbb{Z}^3$ and its RoPE phase vector $\boldsymbol{R}_i \in \mathbb{C}^3$ exactly. Use $e^{\mathrm{i}\phi} = \cos\phi + \mathrm{i}\sin\phi$.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# RotaryPositionEmbedding: phases — answer

**Answer**:

# RotaryPositionEmbedding: phases — answer

**Answer**: The image token has $p_f = M = 1$, $p_h = u - \lceil h/2 \rceil = 1 - 1 = 0$, and $p_w = v - \lceil w/2 \rceil = 0 - 1 = -1$. Thus

$$\boldsymbol{p}_i = (1, 0, -1) \in \mathbb{Z}^3.$$

For each axis $d_a = 2$, so only $j = 0$ occurs and $\theta_0^{(a)} = 10000^0 = 1$. Consequently,

$$\boldsymbol{R}_i = \begin{pmatrix} e^{\mathrm{i}} \\ e^0 \\ e^{-\mathrm{i}} \end{pmatrix} = \begin{pmatrix} \cos 1 + \mathrm{i} \sin 1 \\ 1 \\ \cos 1 - \mathrm{i} \sin 1 \end{pmatrix} \in \mathbb{C}^3.$$

In real-and-imaginary notation, this has shape $\mathbb{R}^{3,2}$.

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

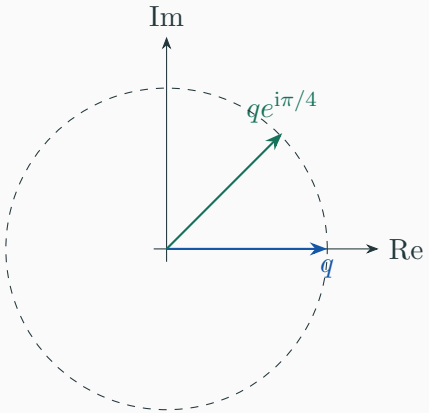
# Applying RoPE to query and key

For a head vector $\boldsymbol{q}_i \in \mathbb{R}^{128}$, pair adjacent coordinates and interpret them as 64 complex numbers $q^{\mathbb{C}}_{i,j} = q_{i,2j} + \mathrm{i}q_{i,2j+1}$. With $R_{i,j} \in \mathbb{C}$ of modulus 1,

$$\widetilde{q}^{\mathbb{C}}_{i,j} = q^{\mathbb{C}}_{i,j} R_{i,j}, \qquad \widetilde{k}^{\mathbb{C}}_{i,j} = k^{\mathbb{C}}_{i,j} R_{i,j}.$$

The equivalent real rotation is

$$\begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}.$$




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Applying RoPE to query and key — exercise

**Exercise**

*For $\boldsymbol{q} = (1, 0)^\top$, $\boldsymbol{k} = (0, 1)^\top$, both in $\mathbb{R}^2$, rotate both by $\phi = \pi/2$. Compute the new vectors and their dot product.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Applying RoPE to query and key — answer

**Answer**:

# Applying RoPE to query and key — answer

**Answer**: The matrix is $\boldsymbol{R} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$. Thus $\widetilde{\boldsymbol{q}} = (0,1)^\top$, $\widetilde{\boldsymbol{k}} = (-1,0)^\top$ and $\widetilde{\boldsymbol{q}}^\top \widetilde{\boldsymbol{k}} = 0$.

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# SingleStreamTransformerBlock

Input $\underline{\boldsymbol{X}} \in \mathbb{R}^{L,4096}$; shared $\underline{\boldsymbol{P}} \in \mathbb{R}^{2,16384}$; RoPE $\underline{\boldsymbol{R}} \in \mathbb{C}^{L,64}$.

For each token, divide the selected modulation into $(\boldsymbol{s}_1, \boldsymbol{g}_1, \boldsymbol{s}_2, \boldsymbol{g}_2)$, all in $\mathbb{R}^{4096}$.

$$\underline{\boldsymbol{U}} = \mathrm{LN}(\underline{\boldsymbol{X}}) \odot (1 + \boldsymbol{s}_1),$$

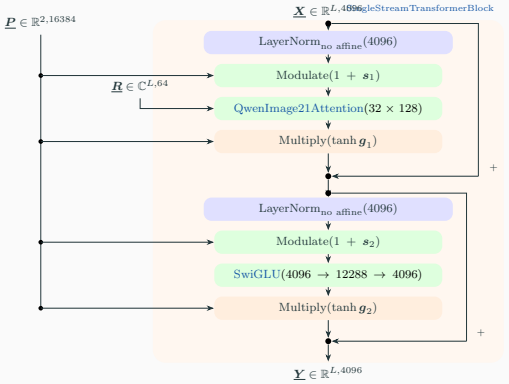
$$\underline{\boldsymbol{Z}} = \underline{\boldsymbol{X}} + \tanh(\boldsymbol{g}_1) \odot \mathrm{Attention}(\underline{\boldsymbol{U}}, \underline{\boldsymbol{R}}),$$

$$\underline{\boldsymbol{V}} = \mathrm{LN}(\underline{\boldsymbol{Z}}) \odot (1 + \boldsymbol{s}_2),$$

$$\underline{\boldsymbol{Y}} = \underline{\boldsymbol{Z}} + \tanh(\boldsymbol{g}_2) \odot \mathrm{SwiGLU}(\underline{\boldsymbol{V}}).$$

Two additive skip connections; one shared modulation projection is reused across all 32 blocks. All arrows carry $4096$-dimensional tokens.

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

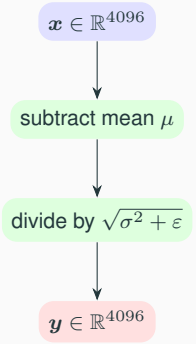


# LayerNorm without affine parameters

For $\boldsymbol{x} \in \mathbb{R}^{4096}$, let $\mu = \frac{1}{4096}\sum_j x_j$, $\sigma^2 = \frac{1}{4096}\sum_j (x_j - \mu)^2$.

$$y_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + 10^{-6}}}, \qquad \boldsymbol{y} \in \mathbb{R}^{4096}.$$

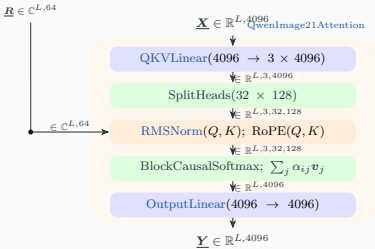
Qwen's two in-block LayerNorms have no learnable $\gamma$ or $\beta$. This differs from the affine LayerNorm used in SD1.5's BasicTransformerBlock.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# QwenImage21Attention

Each token $\boldsymbol{x}_i \in \mathbb{R}^{4096}$ is projected by learned bias-free matrices into query, key and value vectors. Split into 32 heads of 128 dimensions. Q and K receive RMSNorm and the same positional RoPE; V does not. Block-causal softmax attention then gives 32 weighted sums, concatenated and projected back to 4096. Final linear projection is bias-free; dropout=0. The output sequence has shape $\mathbb{R}^{L,4096}$.



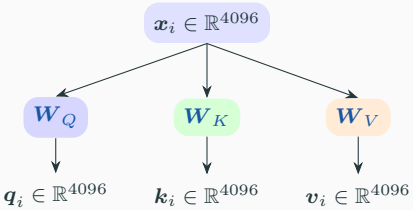

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# QKVLinear in QwenImage21Attention

Let the $L$ token vectors be $\boldsymbol{x}_i \in \mathbb{R}^{4096}$. The three bias-free learned maps are

$$\boldsymbol{q}_i = \boldsymbol{W}_Q \boldsymbol{x}_i, \quad \boldsymbol{k}_i = \boldsymbol{W}_K \boldsymbol{x}_i, \quad \boldsymbol{v}_i = \boldsymbol{W}_V \boldsymbol{x}_i,$$

with $\boldsymbol{W}_Q, \boldsymbol{W}_K, \boldsymbol{W}_V \in \mathbb{R}^{4096,4096}$. Each output vector is in $\mathbb{R}^{4096}$. Collecting the token rows and Q/K/V axis gives $\underline{\boldsymbol{T}} \in \mathbb{R}^{L,3,4096}$.



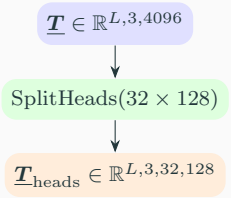

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# SplitHeads in QwenImage21Attention

The output of QKVLinear has shape $\underline{\boldsymbol{T}} \in \mathbb{R}^{L,3,4096}$. Each 4096-dimensional Q/K/V vector is split into 32 successive blocks of 128 coordinates:

$$\boldsymbol{q}_i = \begin{pmatrix} \boldsymbol{q}_i^{(1)} \\ \vdots \\ \boldsymbol{q}_i^{(32)} \end{pmatrix}, \quad \boldsymbol{k}_i = \begin{pmatrix} \boldsymbol{k}_i^{(1)} \\ \vdots \\ \boldsymbol{k}_i^{(32)} \end{pmatrix}, \quad \boldsymbol{v}_i = \begin{pmatrix} \boldsymbol{v}_i^{(1)} \\ \vdots \\ \boldsymbol{v}_i^{(32)} \end{pmatrix},$$

where each head vector belongs to $\mathbb{R}^{128}$. The reshaped array belongs to $\mathbb{R}^{L,3,32,128}$. Splitting heads is parameter-free.



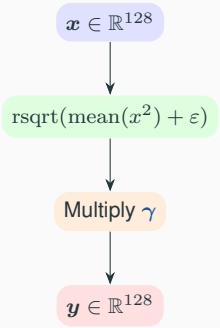

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Q/K RMSNorm in attention

After Q/K projection and head splitting, $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{128}$. For $\boldsymbol{x} \in \mathbb{R}^{128}$,

$$y_j = \gamma_j \frac{x_j}{\sqrt{\frac{1}{128}\sum_{a=1}^{128} x_a^2 + 10^{-6}}}, \quad \boldsymbol{y} \in \mathbb{R}^{128}.$$

The scale $\gamma \in \mathbb{R}^{128}$ is learned (separately for Q and K), and no mean is subtracted. This is not the zero-centered storage convention of TextProjection.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Q/K RMSNorm in attention — exercise

**Exercise**

*For a toy head $\boldsymbol{x} = (3, 4)^\top$, $\varepsilon = 0$ and $\boldsymbol{\gamma} = (2, 1)^\top$, compute the RMS-normalized vector.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Q/K RMSNorm in attention — answer

**Answer**:

# Q/K RMSNorm in attention — answer

**Answer**: $\mathrm{mean}(x^2) = (9+16)/2 = 25/2$. Hence $\boldsymbol{y} = \left(6\sqrt{2}/5, 4\sqrt{2}/5\right)^\top \in \mathbb{R}^2$.

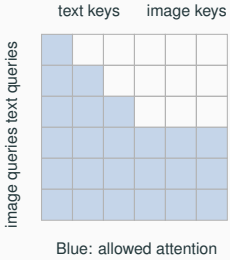
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Block-causal attention mask

Let $i, j \in \{1, \ldots, L\}$ denote query/key positions and let $b_i$ be the image-block id ($b_i = -1$ for text). Define the allowed-key indicator

$$A_{ij} = \begin{cases} 1, & j \le i \text{ or } (b_i = b_j \ge 0), \\ 0, & \text{otherwise.} \end{cases}$$

The attention score $s_{ij}$ is excluded if $A_{ij} = 0$. Within each image block, all spatial tokens can attend to one another. Text positions are causal. For text-to-image, target-image queries can attend to the entire prompt and target image.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Block-causal attention mask — exercise

**Exercise**

*For two text tokens followed by two target-image tokens, list the allowed key positions for queries $i = 1, 2, 3, 4$. Use one image-block id for positions 3 and 4.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Block-causal attention mask — answer

**Answer**:

# Block-causal attention mask — answer

**Answer**: Queries 1 and 2 allow keys $\{1\}$ and $\{1, 2\}$. Image queries 3 and 4 both allow $\{1, 2, 3, 4\}$, since all target-image tokens lie in the same bidirectional block.

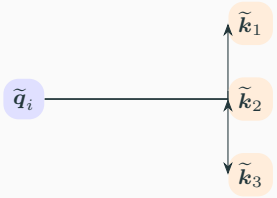
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# QwenImage21Attention: scores and weights

For each head $r \in \{1, \dots, 32\}$ and token $i$, learned projections produce $\boldsymbol{q}_i^{(r)}, \boldsymbol{k}_i^{(r)}, \boldsymbol{v}_i^{(r)} \in \mathbb{R}^{128}$. Apply learned RMSNorm (Q/K only) and RoPE to get $\widetilde{\boldsymbol{q}}_i^{(r)}, \widetilde{\boldsymbol{k}}_i^{(r)} \in \mathbb{R}^{128}$.

$$s_{ij}^{(r)} = \frac{(\widetilde{\boldsymbol{q}}_i^{(r)})^\top \widetilde{\boldsymbol{k}}_j^{(r)}}{\sqrt{128}}, \quad \alpha_{ij}^{(r)} = \frac{A_{ij} e^{s_{ij}^{(r)}}}{\sum_\ell A_{i\ell} e^{s_{i\ell}^{(r)}}}.$$

$\alpha_{ij}^{(r)} \in \mathbb{R}_{\geq 0}$; $\sum_j \alpha_{ij}^{(r)} = 1$.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# QwenImage21Attention: weights — exercise

**Exercise**

*For one toy attention head with $d_h = 2$, take $M = 3$ text tokens and $N = 2$ target-image tokens. At image query position $i = 4$, all five keys are allowed by the block-causal mask. At text query position $i = 2$, only keys $j = 1, 2$ are allowed. Suppose the already normalized and RoPE-rotated vectors are*

$$\widetilde{\boldsymbol{q}}_2 = \widetilde{\boldsymbol{q}}_4 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad (\widetilde{\boldsymbol{k}}_1, \ldots, \widetilde{\boldsymbol{k}}_5) = \left( \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} -1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} -1 \\ 0 \end{pmatrix} \right).$$

*For both $i = 4$ and $i = 2$, find the five dot products, masked scaled scores and softmax weights to four decimal places.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# QwenImage21Attention: weights — answer

**Answer**:

## QwenImage21Attention: weights — answer

**Answer**: The five dot products are $(1, -1, 0, 1, -1)$. Dividing by $\sqrt{d_h} = \sqrt{2}$, the scores are

$$\boldsymbol{s} = \frac{1}{\sqrt{2}}(1, -1, 0, 1, -1)^\top \in \mathbb{R}^5.$$

The normalizing denominator is

$$D = 2e^{1/\sqrt{2}} + 2e^{-1/\sqrt{2}} + 1 \approx 6.0424.$$

Thus

$$\boldsymbol{\alpha} = \frac{1}{D}\begin{pmatrix} e^{1/\sqrt{2}} \\ e^{-1/\sqrt{2}} \\ 1 \\ e^{1/\sqrt{2}} \\ e^{-1/\sqrt{2}} \end{pmatrix} \approx \begin{pmatrix} 0.3356 \\ 0.0816 \\ 0.1655 \\ 0.3356 \\ 0.0816 \end{pmatrix} \in \mathbb{R}^5.$$

For the text query $i = 2$, keys $j = 3, 4, 5$ are masked out even though their unmasked dot products remain $(0, 1, -1)$. The masked scores are

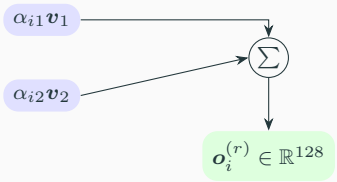
$$\boldsymbol{s}_2 = \left(\tfrac{1}{\sqrt{2}}, -\tfrac{1}{\sqrt{2}}, -\infty, -\infty, -\infty\right)^\top.$$

# QwenImage21Attention: weighted values

After the scores are normalized, each head returns

$$\boldsymbol{o}_i^{(r)} = \sum_{j:A_{ij}=1} \alpha_{ij}^{(r)} \boldsymbol{v}_j^{(r)} \in \mathbb{R}^{128}.$$

Concatenate 32 head vectors to $\boldsymbol{o}_i \in \mathbb{R}^{4096}$, then output $\boldsymbol{y}_i = \boldsymbol{W}_O \boldsymbol{o}_i \in \mathbb{R}^{4096}$, with $\boldsymbol{W}_O \in \mathbb{R}^{4096,4096}$ and no bias.




32 heads $\rightarrow \boldsymbol{W}_O$

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# QwenImage21Attention: weighted values — exercise

**Exercise**

*Let three allowed keys have attention weights* $\alpha_1 = \frac{1}{2}$, $\alpha_2 = \frac{1}{3}$, $\alpha_3 = \frac{1}{6}$. *Given*

$$\boldsymbol{v}_1 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad \boldsymbol{v}_2 = \begin{pmatrix} 0 \\ 3 \end{pmatrix}, \quad \boldsymbol{v}_3 = \begin{pmatrix} 6 \\ -6 \end{pmatrix} \in \mathbb{R}^2,$$

*compute the attention output* $\boldsymbol{o} = \sum_{j=1}^{3} \alpha_j \boldsymbol{v}_j$.

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# QwenImage21Attention: weighted values — answer

**Answer**:

# QwenImage21Attention: weighted values — answer

**Answer**: The three weighted values are

$$\tfrac{1}{2}\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \tfrac{1}{3}\boldsymbol{v}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \tfrac{1}{6}\boldsymbol{v}_3 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Therefore $\boldsymbol{o} = (1+0+1,\ 0+1-1)^\top = (2,0)^\top \in \mathbb{R}^2$.

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

## Pros and cons of convolutional layer

In the single stream attention, a pixel can gather global information in the latent image, and also the user input information at once.

## Pros and cons of convolutional layer

In the single stream attention, a pixel can gather global information in the latent image, and also the user input information at once.

One concern is the computational complexity. It may be $O((hw)^4)$.

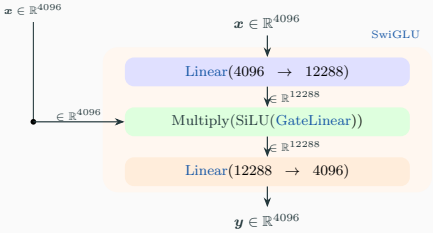
# SwiGLU

For $\boldsymbol{x} \in \mathbb{R}^{4096}$, the 3 bias-free learned matrices are

$$\boldsymbol{A}, \boldsymbol{G} \in \mathbb{R}^{12288,4096}, \quad \boldsymbol{B} \in \mathbb{R}^{4096,12288}.$$

$$\boldsymbol{y} = \boldsymbol{B}\left[\left(\boldsymbol{A}\boldsymbol{x}\right) \odot \mathrm{SiLU}\left(\boldsymbol{G}\boldsymbol{x}\right)\right] \in \mathbb{R}^{4096}.$$

This is the MLP in each of the 32 transformer blocks.



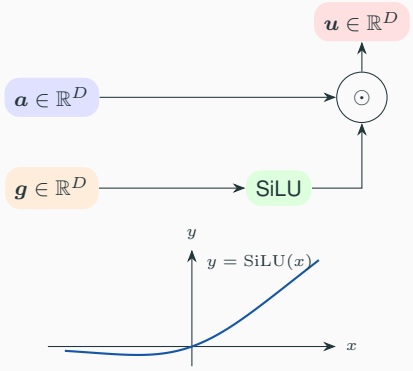

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# SwiGLU: elementwise gated product

For $\boldsymbol{a}, \boldsymbol{g} \in \mathbb{R}^D$, define the gated product

$$u_j = a_j \ \mathrm{SiLU}(g_j) = a_j \frac{g_j}{1 + e^{-g_j}}, \quad j = 1, \ldots, D.$$

Then $\boldsymbol{u} \in \mathbb{R}^D$. The projections creating $\boldsymbol{a}, \boldsymbol{g}$ are learned, while the product and SiLU themselves are not.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# SwiGLU: elementwise gated product — exercise

**Exercise**

*For a toy gate* $\boldsymbol{a} = (2, 3)^\top$ *and* $\boldsymbol{g} = (0, \ln 2)^\top$*, calculate* $\boldsymbol{a} \odot \mathrm{SiLU}(\boldsymbol{g})$*.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# SwiGLU: elementwise gated product — answer

**Answer**:

# SwiGLU: elementwise gated product — answer

**Answer**: $\mathrm{SiLU}(0) = 0$, and $\mathrm{SiLU}(\ln 2) = 2\ln 2/3$. Thus $\boldsymbol{u} = (0,\ 3(2\ln 2/3))^\top = (0, 2\ln 2)^\top$.

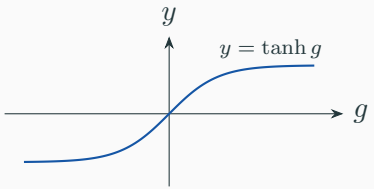
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Tanh-gated residual addition

For $\boldsymbol{x}, \boldsymbol{a}, \boldsymbol{g} \in \mathbb{R}^d$, define

$$\boldsymbol{y} = \boldsymbol{x} + \tanh(\boldsymbol{g}) \odot \boldsymbol{a}, \qquad \tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

The residual sum keeps the shape $\mathbb{R}^d$. $\boldsymbol{g}$ comes from learned shared modulation, but $\tanh$ and addition are parameter-free.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Tanh-gated residual addition — exercise

**Exercise**

*Let* $\boldsymbol{x} = (1, 2)^\top$*,* $\boldsymbol{a} = (2, -2)^\top$*, and* $\boldsymbol{g} = (0, \frac{1}{2}\ln 3)^\top$*. Find* $\boldsymbol{y} = \boldsymbol{x} + \tanh(\boldsymbol{g}) \odot \boldsymbol{a}$*.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Tanh-gated residual addition — answer

**Answer**:

# Tanh-gated residual addition — answer

**Answer**: $\tanh(0) = 0$ and $\tanh\left(\frac{1}{2}\ln 3\right) = (3-1)/(3+1) = 1/2$. Thus $\boldsymbol{y} = (1,2)^\top + (0,-1)^\top = (1,1)^\top$.

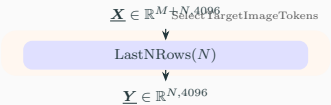
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# SelectTargetImageTokens

Let $L = M + N$, and input the final joint token sequence $\underline{\boldsymbol{X}} \in \mathbb{R}^{L,4096}$. In text-to-image, target-image tokens are the last $N = hw$ rows.

$$\boldsymbol{y}_i = \boldsymbol{x}_{M+i} \in \mathbb{R}^{4096}, \quad 1 \le i \le N.$$

$\underline{\boldsymbol{Y}} \in \mathbb{R}^{N,4096}$; this deterministic row selection has no learned parameters. With condition images, select according to the target-token mask.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# SelectTargetImageTokens — exercise

## Exercise

*Let $M = 3$, $N = 4$, $d = 3$ and let the joint sequence consist of three text rows followed by four image rows:*

$$\underline{\boldsymbol{X}} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \\ 13 & 14 & 15 \\ 16 & 17 & 18 \\ 19 & 20 & 21 \end{pmatrix} \in \mathbb{R}^{7,3}.$$

*Find $\underline{\boldsymbol{Y}} = \mathrm{SelectTargetImageTokens}(\underline{\boldsymbol{X}})$ and its shape.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# SelectTargetImageTokens — answer

**Answer**:

# SelectTargetImageTokens — answer

**Answer**: The first $M = 3$ text rows are discarded. The last $N = 4$ image-token rows remain:

$$\underline{\boldsymbol{Y}} = \begin{pmatrix} 10 & 11 & 12 \\ 13 & 14 & 15 \\ 16 & 17 & 18 \\ 19 & 20 & 21 \end{pmatrix} \in \mathbb{R}^{4,3}.$$

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

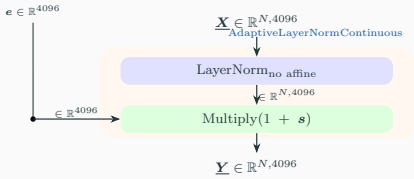
# AdaptiveLayerNormContinuous

Input $\underline{\boldsymbol{X}} \in \mathbb{R}^{N,4096}$, time embedding $\underline{\boldsymbol{E}} \in \mathbb{R}^{2,4096}$. For target tokens use the target-time row $\boldsymbol{e} \in \mathbb{R}^{4096}$:

$$\boldsymbol{s} = \boldsymbol{A}\,\mathrm{SiLU}(\boldsymbol{e}) \in \mathbb{R}^{4096},$$

$$\boldsymbol{y}_i = \mathrm{LayerNorm}_{\text{no affine}}(\boldsymbol{x}_i) \odot (\mathbf{1} + \boldsymbol{s}) \in \mathbb{R}^{4096}.$$

One learned bias-free matrix $\boldsymbol{A} \in \mathbb{R}^{4096,4096}$. Only scale, no shift.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# AdaptiveLayerNormContinuous — exercise

**Exercise**

*Suppose a two-feature token has normalized vector $(1, -1)^\top$ and its selected learned scale is $s = (1, -1/2)^\top$. Calculate the adaptive-normalization output.*

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# AdaptiveLayerNormContinuous — answer

**Answer**:

# AdaptiveLayerNormContinuous — answer

**Answer**: $\boldsymbol{y} = (1, -1)^\top \odot (2, 1/2)^\top = (2, -1/2)^\top \in \mathbb{R}^2$.

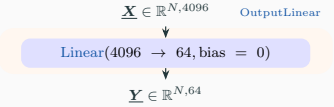
Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# OutputLinear

The final learned projection maps each target token from 4096 to 64 components:

$$\boldsymbol{y}_i = \boldsymbol{W}_{\text{out}}\boldsymbol{x}_i, \quad \boldsymbol{W}_{\text{out}} \in \mathbb{R}^{64,4096}.$$

No bias. For $N = hw$ tokens, $\underline{\boldsymbol{X}} \in \mathbb{R}^{N,4096}$ becomes $\underline{\boldsymbol{Y}} \in \mathbb{R}^{N,64}$.



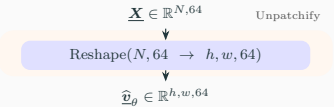

Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Unpatchify

Since $p = 1$, there is one 64-dimensional output token for each position on the $h \times w$ latent grid. For zero-based indices $0 \le u < h$, $0 \le v < w$,

$$\boldsymbol{y}_{u,v} = \boldsymbol{x}_{uw+v} \in \mathbb{R}^{64}.$$

The inverse of patchification reshapes $\underline{\boldsymbol{X}} \in \mathbb{R}^{N,64}$ to $\underline{\boldsymbol{Y}} \in \mathbb{R}^{h,w,64}$, where $N = hw$. There are no learnable parameters.




Source: Diffusers QwenImage21Transformer2DModel; `transformer_qwenimage21.py`.

# Outline

1. Introduction

2. Mathematical notation

3. What are the necessary conditions for NNs in iterative generation?

4. How does Stable Diffusion 1.5's U-Net work?

**5. Diffusion Transformer of Qwen-Image-2.1**

6. Summary

# Outline

1. Introduction

2. Mathematical notation

3. What are the necessary conditions for NNs in iterative generation?

4. How does Stable Diffusion 1.5's U-Net work?

5. Diffusion Transformer of Qwen-Image-2.1

**6. Summary**

# Summary

## Summary

- In iterative generation for visual data generation, the input and output shapes of the NN must be identical. Also, it must accepts a variable shape input to control the output shape.

## Summary

- In iterative generation for visual data generation, the input and output shapes of the NN must be identical. Also, it must accepts a variable shape input to control the output shape.
- SD 1.5's U-net integrates the user input information and the latent image information in cross attention, while in Qwen-Image-2.1, a single stream diffusion Transformer integrates them.

## Summary

- In iterative generation for visual data generation, the input and output shapes of the NN must be identical. Also, it must accepts a variable shape input to control the output shape.
- SD 1.5's U-net integrates the user input information and the latent image information in cross attention, while in Qwen-Image-2.1, a single stream diffusion Transformer integrates them.
- SD 1.5's U-net mainly uses a convolutional layer, which is a spacially local operation. Qwen-Image-2.1 uses a single stream diffusion Transformer, which is a specially global operation. Transformer can easily capture the global information, but the computational cost is heavy.

# Implementation references: Stable Diffusion 1.5

- Stable Diffusion 1.5 U-Net config

  https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5/blob/main/unet/config.json

- Stable Diffusion 1.5 VAE config

  https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5/blob/main/vae/config.json

- Diffusers U-Net blocks

  https://github.com/huggingface/diffusers/blob/main/src/diffusers/models/unets/unet_2d_blocks.py

- Diffusers residual blocks

  https://github.com/huggingface/diffusers/blob/main/src/diffusers/models/resnet.py

- Diffusers VAE blocks

  https://github.com/huggingface/diffusers/blob/main/src/diffusers/models/autoencoders/vae.py

# Implementation references: Qwen-Image-2.1

- Qwen DiT configuration

  https://huggingface.co/Qwen/Qwen-Image-2.1/blob/main/transformer/config.json

- Qwen VAE configuration

  https://huggingface.co/Qwen/Qwen-Image-2.1/blob/main/vae/config.json

- Qwen DiT implementation

  https://github.com/huggingface/diffusers/blob/main/src/diffusers/models/transformers/transformer_qwenimage21.py

- Qwen VAE implementation

  https://github.com/huggingface/diffusers/blob/main/src/diffusers/models/autoencoders/autoencoder_kl_qwenimage21.py